۵۳
نظر

آموزش کتابخانه Matplotlib و Seaborn برای رسم نمودار در پایتون

سرفصل‌های مقاله

MatPlotLib چیست؟

Matplotlib PyPlot چیست؟

نحوه ی رسم اشکال دوبعدی در matplotlib

رسم نمودار میله ای در پایتون

رسم نمودار هیستوگرام در پایتون

رسم نمودار پراکندگی در پایتون

رسم نمودار دایره ای در پایتون

رسم نمودار تابع در پایتون

زیرنمودار یا Subplots در پایتون

رسم نمودار سه بعدی در پایتون

آموزش Python Seaborn

تفاوت میان matplotlib و seaborn

وارد کردن کتابخانه‌ها و مجموعه داده

توابع ترسیم نمودار در seaborn

نقش کتابخانه ی Pandas در Seaborn

تم‌های Seaborn : Seaborn themes

جمع بندی

نمایش بیشتر

مصورسازی داده، تجسم سازی داده یا Data Visualization ارائه ی گرافیکی داده است که هدف اصلی آن انتقال بهینه ی اطلاعات به کاربران از طریق نمایش روابط میان داده‌ها به کمک نمودارها است. Matplotlib از کتابخانه‌های معروف پایتون برای ترسیم نمودارهاست. Seaborn از دیگر کتابخانه‌های پایتون برای مصورسازی است که بر پایه ی Matplotlib بنا شده و امکانات بیشتری برای کاربران در جهت ترسیم نمودارها فراهم می‌کند. هر دوی این کتابخانه‌ها جزو کتابخانه‌های بسیار پرکاربرد رسم نمودار و مصور سازی داده در پایتون هستند. از آنجایی که یادگیری پکیج‌های Matplotlib، Pandas، Numpy و Seaborn برای شروع یادگیری داده کاوی با پایتون بسیار ضروری است در این مقاله ضمن معرفی انواع نمودارهای پرکاربرد در مصورسازی داده‌ها به آموزش Matplotlib و در ادامه ی آن به آموزش Seaborn می‌پردازیم.

MatPlotLib چیست؟

Matplotlib از کتابخانه های رسم نمودار در زبان برنامه نویسی پایتون است که به همراه بسیاری از کتابخانه های این زبان که بر مبنای کار با مقادیر عددی مانند NumPy و Pandas توسعه یافته اند به کار گرفته می شود.
Matplotlib برای گنجاندن نمودارها در اپلیکیشن ها با استفاده از ابزارهای گرافیکی پایتون مانند wxPyton، Tkinter و ... API مبتنی بر شی گرایی فراهم می‌کند.
Matplotlib توسط JohnD.Hunter در سال ۲۰۰۳ توسعه داده شد. برای اطلاعات بیشتر به وبسایت آن مراجعه کنید.

Matplotlib PyPlot چیست؟

matplotlib.pyplot مجموعه ای از دستورات و توابعی است که کتابخانه ی matplotlib را قادر می سازد تا همانند زبان برنامه نویسی MATLAB باشد. هر تابع pyplot می‌تواند تغییراتی در شکل نمودار مانند رسم نمودار، ایجاد خطوطی در ناحیه ی ترسیم شده، ایجاد برچسب برای نمودار و ... ایجاد کند. در matplotlib.pyplot حالت‌های مختلف حین فراخوانی توابع حفظ می شود، به طوری که تأثیر مواردی همچون شکل فعلی و ناحیه ی رسم شده حفظ می شود و موارد جدید از تغییرات ناشی از فراخوانی توابع بر روی محورهای فعلی اعمال می‌شود.

در پس زمینه matplotlib چه خبر است؟

Matplotlib موارد استفاده و فرمت‌های خروجی مختلفی را شامل می شود. برخی افراد با استفاده از python shell را برای تعامل با matplotlib استفاده می کنند و شکل نمودار مانند پنجره‌های pop up برای آن‌ها ظاهر می‌شود (مانند دیدن تصاویر نمودار در MATLAB)

برخی هم از JupyterNotebook برای نوشتن کدهای خود استفاده می کنند و نمودارهای حاصل را به طور inline در همان صفحه زیر کدهای خود می‌بینند که این روش باعث تجزیه تحلیل سریع تر و آسان‌تر کدها و نتایج آن‌ها است. برخی هم mathplotlib را به صورت اسکریپت هایی برای تولید تصاویر postscript از شبیه - سازی‌های عددی استفاده می‌کنند.

برای پشتیبانی تمامی این موارد استفاده mathplotlib قادر است انواع مختلف خروجی را تولید کند و همه ی این موارد در پس زمینه یا backend این کتابخانه گنجانده شده است.

نحوه ی رسم اشکال دوبعدی در matplotlib

matplotlib جزء آن دسته از پکیج هایی است که امکانات گرافیکی زیادی برای مصورسازی داده‌ها همچون ترسیمات دوبعدی، سه بعدی و حتی انیمیشن را فراهم می‌کند. در این مقاله به نحوه ی رسم نمودار دوبعدی در پایتون می‌پردازیم. در میان

نمودارهای دوبعدی ترسیم خطوط، نمودار پراکندگی، میله ای، هیستوگرام، دایره ای و سایر نمودارهای پرکاربرد را بررسی می‌کنیم.

رسم خطوط در matplotlib

برای رسم یک خط در کتابخانه ی مت پلات مراحل زیر را داریم:

تعریف محور x و مقادیر متناظر در محور y به صورت لیست‌های جداگانه

رسم آنها بر روی صفحه با تابع plot

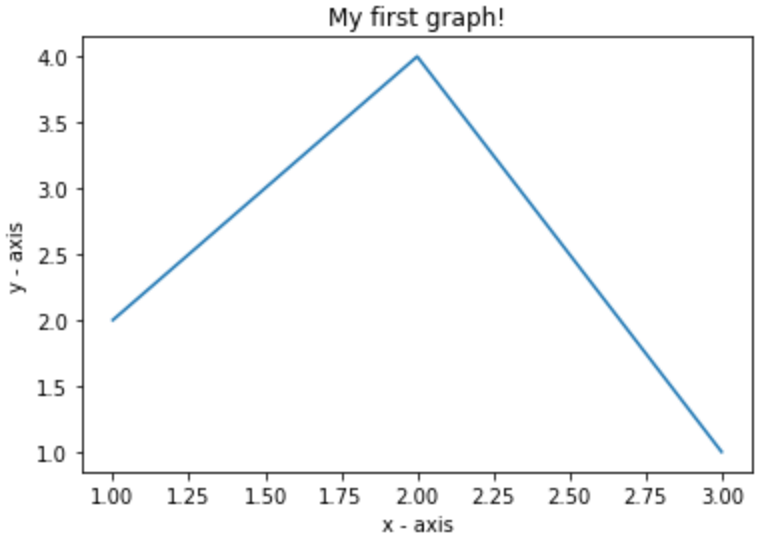
اختصاص نام به محورهای x و y با توابع xlabel و ylabel

دادن عنوان به نمودار با تابع title

در پایان تمامی کدها در matplotlib بکارگیری تابع plt.show برای دیدن شکل نهایی نمودار

```
# importing the required module
import matplotlib.pyplot as plt
# x axis values
x = [1,2,3]
# corresponding y axis values
y = [2,4,1]
# plotting the points
plt.plot(x, y)
# naming the x axis
plt.xlabel('x - axis')
# naming the y axis
plt.ylabel('y - axis')
# giving a title to my graph
plt.title('My first graph!')
# function to show the plot
plt.show()
```

خروجی کد بالا نمودار خطی زیر است:



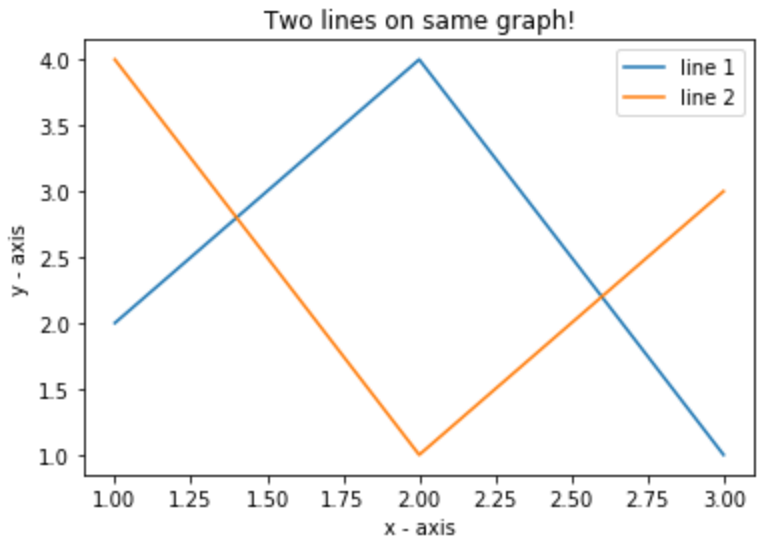
برای رسم دو یا چند خط بر روی یک نمودار مانند کد زیر عمل می‌کنیم که در آن:

همان طور که می‌بینید دو خط را بر روی یک نمودار رسم کرده ایم. برای ایجاد تمایز به هر یک نام (label) متفاوتی داده ایم، که این نام به عنوان آرگومان ورودی تابع plot تعریف می شود.

جعبه ی مستطیل شکل کوچکی که اطلاعاتی در مورد نوع و شکل خطوط می‌دهد legend نامیده می‌شود. برای افزودن legend به نمودار از تابع legend استفاده می-کنیم.

```
import matplotlib.pyplot as plt
# line 1 points
x1 = [1,2,3]
y1 = [2,4,1]
# plotting the line 1 points
plt.plot(x1, y1, label = "line 1")
# line 2 points
x2 = [1,2,3]
y2 = [4,1,3]
# plotting the line 2 points
plt.plot(x2, y2, label = "line 2")
# naming the x axis
plt.xlabel('x - axis')
# naming the y axis
plt.ylabel('y - axis')
# giving a title to my graph
plt.title('Two lines on same graph!')
# show a legend on the plot
plt.legend()
# function to show the plot
plt.show()
```

خروجی کد به شکل زیر است:

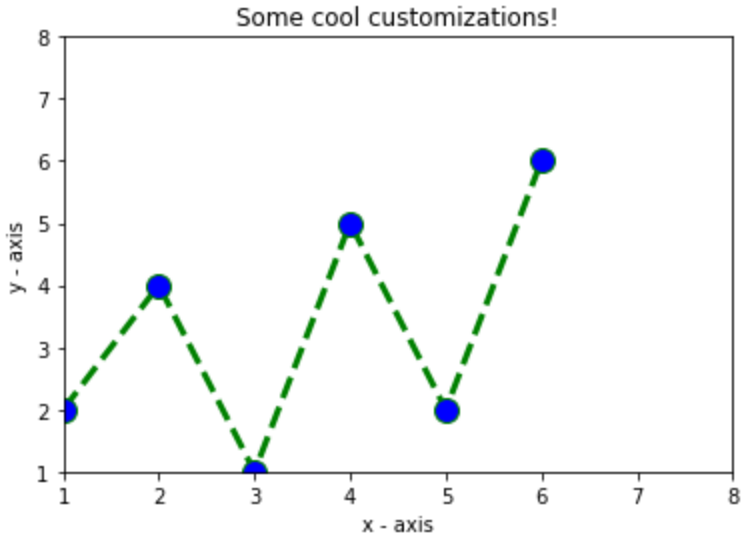


در رسم نمودارها به کمک مت پلات می توان ویژگی های بیشتری را به نمودار افزود.

نمودار حاصل از کد مثال زیر را در نظر بگیرید:

```
import matplotlib.pyplot as plt
# x axis values
x = [1,2,3,4,5,6]
# corresponding y axis values
y = [2,4,1,5,2,6]
# plotting the points
plt.plot(x, y, color='green', linestyle='dashed', linewidth
                marker='o', markerfacecolor='blue', markersi
# setting x and y axis range
plt.ylim(1,8)
plt.xlim(1,8)
# naming the x axis
plt.xlabel('x - axis')
# naming the y axis
plt.ylabel('y - axis')
# giving a title to my graph
plt.title('Some cool customizations!')
# function to show the plot
plt.show()
```

خروجی کد به صورت نقاط متصل شده ی زیر با خطوط دندانه دار به شکل زیر است:



همان طور که در شکل می‌بینید در این نمودار:

عرض، شکل و رنگ خط را مقداردهی کرده ایم.

رنگ و شکل و اندازه ی هر نقطه را به طور دلخواه مشخص کرده ایم. (ویژگی های مرتبط با marker)

رنج محورهای z و y را به دلخواه خود تعیین نموده ایم. اگر رنج دهی به مختصات را خودمان تنظیم نمی‌کردیم ماژول pyplot از ویژگی auto-scale برای تنظیم رنج و مقیاس دهی به محورها استفاده می‌کرد.



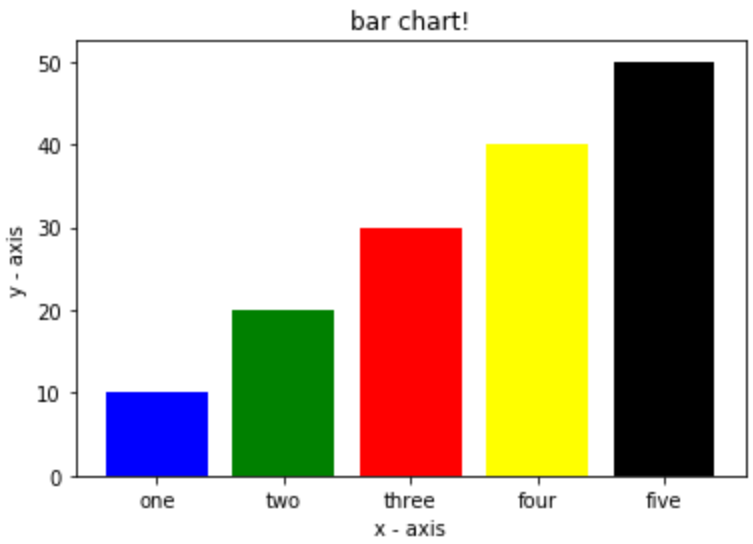
رسم نمودار میله ای در پایتون

نمودار میله ای یا نواری (Bar Plot) نموداری است که داده‌های طبقه بندی شده را با میله‌های مستطیل شکل با ارتفاع یا طول متناسب با مقادیر ارائه شده نشان می‌دهد. میله‌ها می توانند به شکل عمودی یا افقی رسم شوند که گاهی نمودار میله ای عمودی، نمودار خطی نامیده می شود.

در شکل زیر یک نمودار میله ای را به صورت عمودی می‌بینیم:

```
import matplotlib.pyplot as plt
# x-coordinates of left sides of bars
left = [1, 2, 3, 4, 5]
# heights of bars
height = [10, 20, 30, 40, 50]
# labels for bars
tick_label = ['one', 'two', 'three', 'four', 'five']
# plotting a bar chart
plt.bar(left, height, tick_label = tick_label,
        width = 0.8, color = ['blue', 'green', 'red','yellow
# naming the x-axis
plt.xlabel('x - axis')
# naming the y-axis
plt.ylabel('y - axis')
# plot title
plt.title('bar chart!')
# function to show the plot
plt.show()
```

نمودار میله ای حاصل از کد بالا به شکل زیر است:



در این نمودار:

از تابع plt.bar برای رسم نمودار میله ای استفاده می‌کنیم.

مختصات x از سمت چپ به ارتفاع میله‌ها به طور متناظر اختصاص داده می شود.

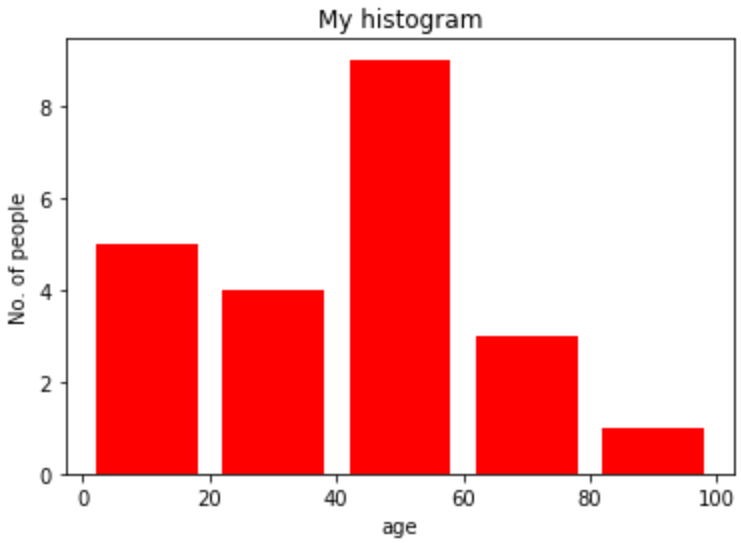
می توانید نامی را به مختصات محور x با تعریف tich_label بدهید.

رسم نمودار هیستوگرام در پایتون

نمودار هیستوگرام یک نمایش دقیق از توزیع داده‌های عددی است. این نمودار تخمینی از توزیع احتمال متغیر پیوسته است و برای اولین بار توسط کارل پیرسون معرفی شده است. تفاوت نمودار هیستوگرام با نمودار میله ای در این است که یک نمودار میله ای رابطه ی دو متغیر را با هم نشان می‌دهد. اما هیستوگرام تنها به یک متغیر مربوط می‌شود.

```
import matplotlib.pyplot as plt
# frequencies
ages = [2,5,70,40,30,45,50,45,43,40,44,
        60,7,13,57,18,90,77,32,21,20,40]
# setting the ranges and no. of intervals
range = (0, 100)
bins = 5
# plotting a histogram
plt.hist(ages, bins, range, color ='red',
        histtype = 'bar', rwidth = 0.8)
# x-axis label
plt.xlabel('age')
# frequency label
plt.ylabel('No. of people')
# plot title
plt.title('My histogram')
# function to show the plot
plt.show()
```

شکل حاصل از کد بالا نمودار هیستوگرامی است که فراوانی تعداد افراد در هر بازه ی سنی روی محور افقی را نمایش می‌دهد:



در این نمودار:

از تابع hist برای رسم هیستوگرام استفاده می کنیم.

تعداد تکرار با لیست ages ارسال شده اند.

رنج و محدوده می‌تواند با تعریف ساختار داده ی تاپل شامل مقادیر مینیمم و ماکسیمم تعیین شود.

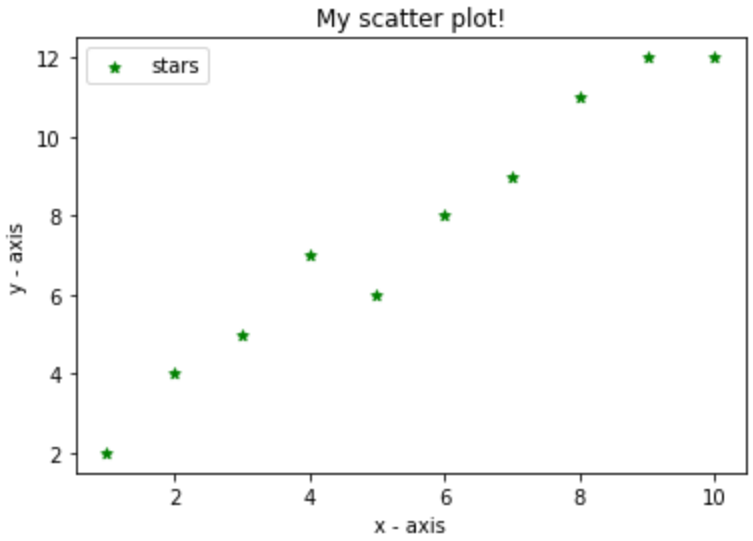
مرحله بعدی اختصاص دادن یا به اصطلاح bin کردن رنج مقادیر (که همان تقسیم کل محدوده بر روی بازه‌ها است.) و سپس شمردن این که چه تعداد از مقادیر در هر بازه وجود دارد. در این مثال مقدار bins را ۱۰ در نظر گرفته ایم پس به طور کلی ۱۰=۱۰۰/۱۰ بازه داریم.

رسم نمودار پراکندگی در پایتون

نمودار پراکندگی یا Scatter Plot نوعی از نمودار است که با استفاده از مختصات دکارتی مقادیر دو متغیر را برای مجموعه ای از اعداد نمایش می‌دهد. در قطعه کد زیر رسم نمودار نقطه ای در پایتون یا همان نمودار پراکندگی را با هم می‌بینیم:

```
import matplotlib.pyplot as plt
# x-axis values
x = [1,2,3,4,5,6,7,8,9,10]
# y-axis values
y = [2,4,5,7,6,8,9,11,12,12]
# plotting points as a scatter plot
plt.scatter(x, y, label= "stars", color= "green",
                        marker= "*", s=30)
# x-axis label
plt.xlabel('x - axis')
# frequency label
plt.ylabel('y - axis')
# plot title
plt.title('My scatter plot!')
# showing legend
plt.legend()
# function to show the plot
plt.show()
```

نمودار حاصل داده‌ها را به شکل نقاط در صفحه مشخص می‌کند و به کاربر اطلاعاتی در مورد نحوه ی پراکندگی داده‌ها می‌دهد:



در این نمودار:

از تابع scatter برای رسم نمودار پراکندگی استفاده می‌کنیم.

مانند یک خط x را متناظر y تعریف می کنیم.

آرگومان marker برای تعیین شکل نقاط روی نمودار استفاده می‌شود که می‌توان با استفاده از پارامتر s سایز آن‌ها را تغییر داد.

رسم نمودار دایره ای در پایتون

نمودار دایره ای یا Pie-Chart یک شکل آماری دایره ای است که برای نشان دادن نسبت و سهم عددی به برش هایی متناظر با هر کدام از آن نسبت‌ها تقسیم می‌شود. در نمودار دایره ای طول قوس هر برش متناسب با کمیتی است که نشان می‌دهد.

```
import matplotlib.pyplot as plt
# defining labels
activities = ['eat', 'sleep', 'work', 'play']
# portion covered by each label
slices = [3, 7, 8, 6]
# color for each label
colors = ['r', 'y', 'g', 'b']
# plotting the pie chart
plt.pie(slices, labels = activities, colors=colors,
                startangle=90, shadow = True, explode = (0,
                radius = 1.2, autopct = '%1.1f%%')
# plotting legend
plt.legend()
# showing the plot
plt.show()
```

نمودار دایره ای به شکل زیر است:



در این نمودار:

نمودار دایره ای را با تابع pie رسم می‌کنیم.

در ابتدا برچسب‌ها را با استفاده از لیستی تحت عنوان activities مشخص کرده-ایم.

سپس سهم هر برچسب را با لیست دیگری با عنوان slices تعیین می کنیم.

رنگ هر برچسب به کمک لیستی با عنوان colors مشخص شده است.

عبارت shadow=True یک حالت سایه مانند زیر هر قوس در دایره ایجاد می‌کند.

صفت startangle نقطه ی شروع نمودار دایره ای را با درجه های معین در جهت عقربه های ساعت از محور x می چرخاند.

Explode برای جداسازی قسمت‌های نمودار و فاصله گرفتن آن‌ها از هم استفاده می‌شود و مقدار فاصله ی قسمت‌های نمودار از هم با توجه به نسبت تعریف شده از اندازه شعاع دایره تعیین می‌شود. مثلاً در شکل بالا سهم سوم به اندازه ی یک دهم اندازه ی شعاع از بقیه جدا شده است.

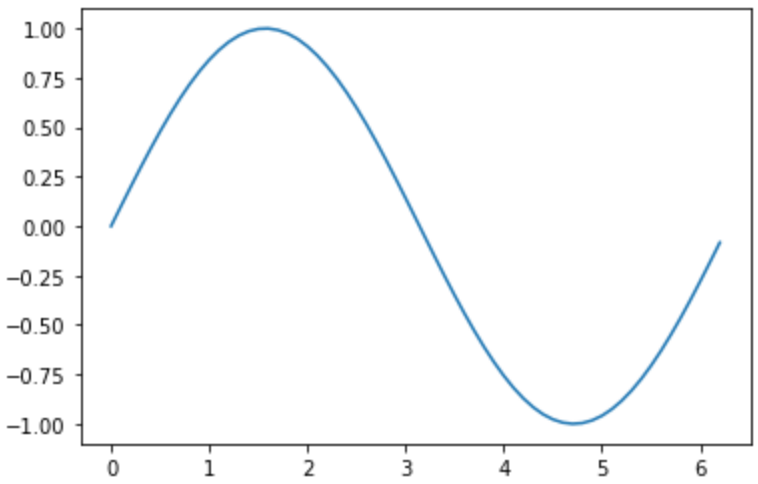
autopct برای شکل دهی مقدار هر برچسب به کار گرفته می‌شود. در این مثال، ما آن را طوری تنظیم کرده ایم تا مقدار درصد هر سهم از نمودار را تا یک رقم اعشار نمایش دهد.

رسم نمودار تابع در پایتون

می توان منحنی معادلات و چند جمله ای با درجه‌های مختلف را ترسیم نمود. برای مثال رسم نمودار سینوسی در پایتون یا همان y=sin(x) که x برحسب رادیان است، مانند کد زیر است:

```
# importing the required modules
import matplotlib.pyplot as plt
import numpy as np
# setting the x - coordinates
x = np.arange(0, 2*(np.pi), 0.1)
# setting the corresponding y - coordinates
y = np.sin(x)
# potting the points
plt.plot(x, y)
# function to show the plot
plt.show()
```

خروجی قطعه کد بالا به شکل زیر است:



در اینجا از NumPy که یک کتابخانه ی عمومی برای کار با اعداد است استفاده کرده - ایم.

برای مقداردهی به متغیرهای محور x از arange استفاده می‌کنیم که دو آرگومان اول آن برای تعیین محدوده ی اعداد و آرگومان سوم فاصله ی بین اعداد در آن بازه است.

برای مقادیر متناظر در محور y از sin استفاده کرده ایم و سینوس مقادیر محور x را محاسبه کرده ایم.

در پایان نمودار را با ارسال آرایه های x و y به تابع plot رسم کرده ایم.

مثال هایی از توابع دیگر همچون رسم نمودار لگاریتمی در پایتون را می‌توان مانند قطعه کد بالا به کمک matplotlib رسم کرد (y=np.log(x)).

زیرنمودار یا Subplots در پایتون

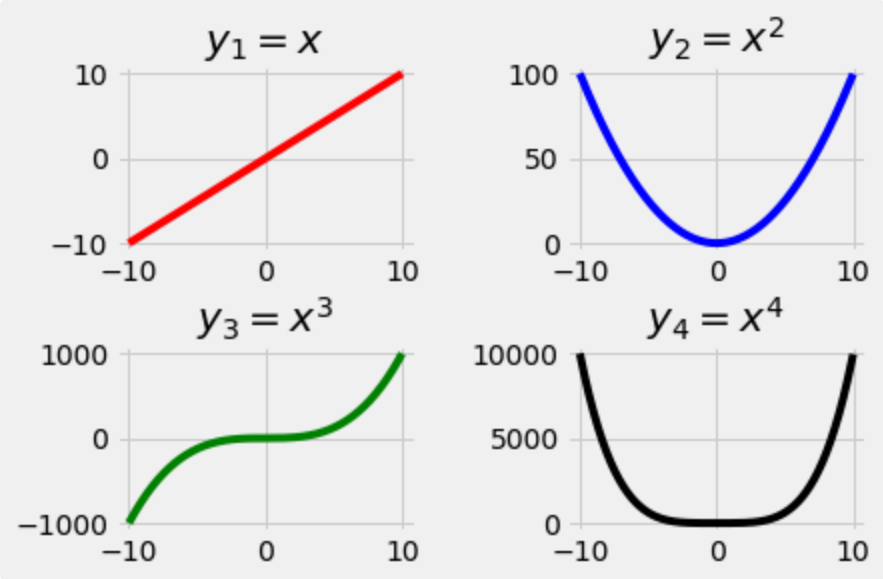
زمانی که می خواهیم دو یا چند نمودار را در یک شکل نمایش دهیم از subplot استفاده می‌کنیم. این کار را با استفاده از دو روش مختلف می‌توان انجام داد:

روش اول رسم زیر نمودار یا Subplot:

```
# importing required modules
import matplotlib.pyplot as plt
import numpy as np
# function to generate coordinates
def create_plot(ptype):
        # setting the x-axis vaues
        x = np.arange(-10, 10, 0.01)
        # setting the y-axis values
        if ptype == 'linear':
                y = x
        elif ptype == 'quadratic':
                y = x**2
        elif ptype == 'cubic':
                y = x**3
        elif ptype == 'quartic':
                y = x**4
        return(x, y)
# setting a style to use
plt.style.use('fivethirtyeight')
# create a figure
fig = plt.figure()
# define subplots and their positions in figure
plt1 = fig.add_subplot(221)
plt2 = fig.add_subplot(222)
plt3 = fig.add_subplot(223)
plt4 = fig.add_subplot(224)
# plotting points on each subplot
x, y = create_plot('linear')
plt1.plot(x, y, color ='r')
plt1.set_title('$y_1 = x$')
x, y = create_plot('quadratic')
plt2.plot(x, y, color ='b')
plt2.set_title('$y_2 = x^2$')
x, y = create_plot('cubic')
plt3.plot(x, y, color ='g')
plt3.set_title('$y_3 = x^3$')
x, y = create_plot('quartic')
```

```
plt4.plot(x, y, color ='k')
plt4.set_title('$y_4 = x^4$')
# adjusting space between subplots
fig.subplots_adjust(hspace=.5,wspace=0.5)
# function to show the plot
plt.show()
```

حاصل چهار زیرنمودار به شکل زیر است:



کد بالا را مرحله به مرحله بررسی می‌کنیم:

plt.style.use('fivethirtyeight'):

شکل نمودارها با تنظیم استایل‌های مختلف موجود یا دلخواه خود می‌توانند تنظیم شوند.

fig = plt.figure( )

این ویژگی به عنوان یک ظرف سطح بالا برای تمامی عناصر نمودار عمل می کند. بنابراین ما یک شکلی تحت عنوان fig تعریف می کنیم که همه ی عناصر زیرنمودارها را در بر بگیرد.

از متد add_subplot برای تعریف زیرنمودارها و تعیین مکان آن‌ها استفاده می‌کنیم. قاعده ی کلی تعریف این تابع به صورت زیر است:

add_subplot(nrows, ncols, plot_number)

plt۱ = fig.add_subplot(۲۲۱)[info]

plt۲ = fig.add_subplot(۲۲۲)

plt۳ = fig.add_subplot(۲۲۳)

plt۴ = fig.add_subplot(۲۲۴)[/info]

اگر یک زیر نمودار در شکلی وجود داشته باشد، می توان شکل را در وضعیتی تصور کرد که به سطرها و ستونهایی تقسیم شده است. (یعنی nrows برای سطرها و ncols برای ستون ها) پارامتر plot_number بیانگر زیرنموداری است که تابع آن را فراخوانی می‌کند تا آن زیرنمودار را رسم کند. Plot_number می تواند محدوده ای از عدد یک تا ماکسیمم میان مقادیر nrows و ncols را داشته باشد. اگر مقادیر این سه پارامتر کمتر از ۱۰ باشند، تابع subplot می تواند تنها با یک مقدار پارامتر عددی صحیح فراخوانی شود که رقم هزارگان این عدد بیانگر تعداد سطرها یا همان nrows ، رقم دهگان نشان دهنده ی تعداد ستون‌ها یا همان ncols و رقم یکان بیانگر مقدار پارامتر plot_number است. این به این معنی است که به جای subplot(۲,۳,۴) می-توان نوشت subplot(۲۳۴) .

x, y = create_plot('linear')

plt۱.plot(x, y, color ='r')

plt۱.set_title('$y_۱ = x$')

می خواهیم نقاط هر زیرنمودار را رسم کنیم. ابتدا محورهای x و y را با تابع create_plot و تعیین نوع بردار مورد نظر، ایجاد می‌کنیم. سپس نقاط روی زیرنمودار را با متد .plot رسم می‌کنیم. عنوان subplot را هم با متد set_title ایجاد می‌کنیم. با استفاده از $ در آغاز و پایان عنوان مورد نظرمان این اطمینان را حاصل می کنیم که در متن عنوان نمودار علامت underscore یا "_" به شکل اندیس یا همان subscript و علامت "^" برای توان یا همان superscript خوانده می‌شوند.

fig.subplots_adjust(hspace=.۵,wspace=۰.۵)

این متد برای ایجاد فاصله میان زیرنمودارها به کارگرفته می‌شود.

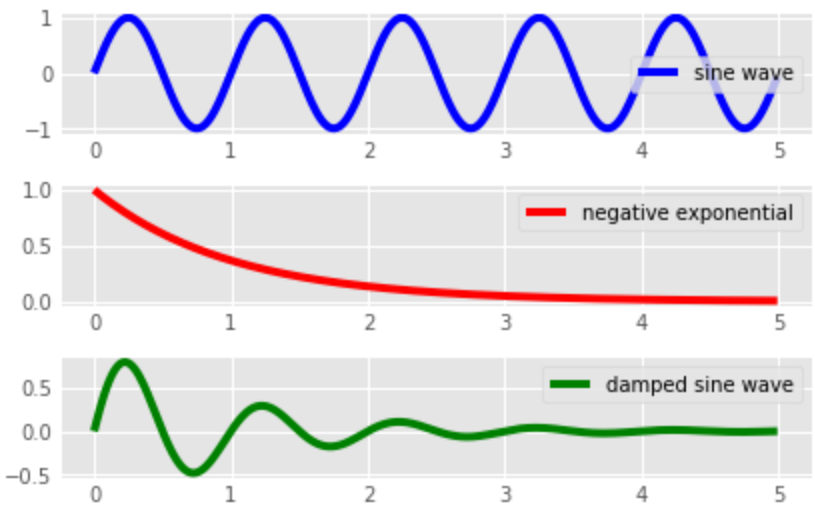
plt.show: مثل همیشه این متد را برای نمایش شکل نهایی نمودار می‌نویسیم.

روش دوم رسم زیر نمودار با Subplot:

```
# importing required modules
import matplotlib.pyplot as plt
import numpy as np
# function to generate coordinates
def create_plot(ptype):
        # setting the x-axis vaues
        x = np.arange(0, 5, 0.01)
        # setting y-axis values
        if ptype == 'sin':
                # a sine wave
                y = np.sin(2*np.pi*x)
        elif ptype == 'exp':
                # negative exponential function
                y = np.exp(-x)
        elif ptype == 'hybrid':
                # a damped sine wave
                y = (np.sin(2*np.pi*x))*(np.exp(-x))
        return(x, y)
# setting a style to use
plt.style.use('ggplot')
# defining subplots and their positions
plt1 = plt.subplot2grid((11,1), (0,0), rowspan = 3, colspan
plt2 = plt.subplot2grid((11,1), (4,0), rowspan = 3, colspan
plt3 = plt.subplot2grid((11,1), (8,0), rowspan = 3, colspan
# plotting points on each subplot
x, y = create_plot('sin')
plt1.plot(x, y, label = 'sine wave', color ='b')
x, y = create_plot('exp')
plt2.plot(x, y, label = 'negative exponential', color = 'r')
x, y = create_plot('hybrid')
plt3.plot(x, y, label = 'damped sine wave', color = 'g')
# show legends of each subplot
plt1.legend()
plt2.legend()
plt3.legend()
```

```
# function to show plot
plt.show()
```

نمودار حاصل به شکل زیر است:



بیایید به بررسی قسمت‌های مهم این کد بپردازیم:

قسمت زیر از کد را ملاحظه نمایید:

plt۱ = plt.subplot۲grid((۱۱,۱), (۰,۰), rowspan = ۳, colspan = ۱)

plt۲ = plt.subplot۲grid((۱۱,۱), (۴,۰), rowspan = ۳, colspan = ۱)

plt۳ = plt.subplot۲grid((۱۱,۱), (۸,۰), rowspan = ۳, colspan = ۱)

Subplot۲grid مشابه pyplot.subplot است اما از اندیس گذاری مبتنی بر صفر استفاده می کند و این امکان را فراهم می‌کند که زیرنمودار چندین خانه را اشغال کند.

توضیح آرگومان‌های این متد به شرح زیر است:

آرگومان اول شکل شبکه ی زیرنمودار را مشخص می کند. (Geometry of the grid)

آرگومان دوم محل زیرنمودار در شبکه را تعیین می کند.

آرگومان سوم تعداد سطرهای پوشیده شده با زیرنمودار است.

آرگومان چهارم تعداد ستون های دربرگرفته توسط زیرنمودار است.

در مثال ما هر زیرنمودار سه سطر و یک ستون را با دو سطر خالی (سطرهای شماره ی ۴ و ۸) پوشش می‌دهد.

توجه: پس از بررسی دو مثال بالا متوجه شدیم که از متد subplot زمانی استفاده می-کنیم که اندازه ی نمودارها یکسان باشد و این در حالی است که متد subplot۲grid انعطاف بیشتری بر روی مکان و اندازه ی زیرنمودارها در اختیار ما قرار می‌دهد.

رسم نمودار سه بعدی در پایتون

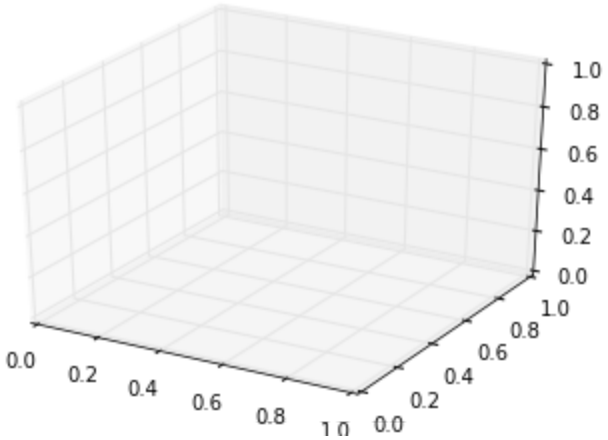
Matpliotlib در ابتدا برای ترسیمات دو بعدی طراحی شده بود. پس از مدتی برخی از ابزارهای ترسیم سه بعدی برمبنای ترسیمات دو بعد این کتابخانه ترسیم شد و نتیجه ی آن مجموعه ای مناسب (اگرچه کمی محدود) از ابزارها برای ترسیمات سه بعدی است. امکان ترسیمات سه بعدی با ابزار toolkit و matplotlib فراهم می‌شود.

```
from mpl_toolkits import mplot3d
```

پس از وارد کردن ماژول mplot۳d به کمک کلمه ی کلیدی 'projection='۳d می‌توان محورهای سه بعدی را کنار توابع معمول رسم محورها، ترسیم نمود. به کد زیر و خروجی آن توجه کنید:

```
import numpy as np
import matplotlib.pyplot as plt
fig = plt.figure()
ax = plt.axes(projection='3d')
```

خروجی نمودار به شکل زیر است:

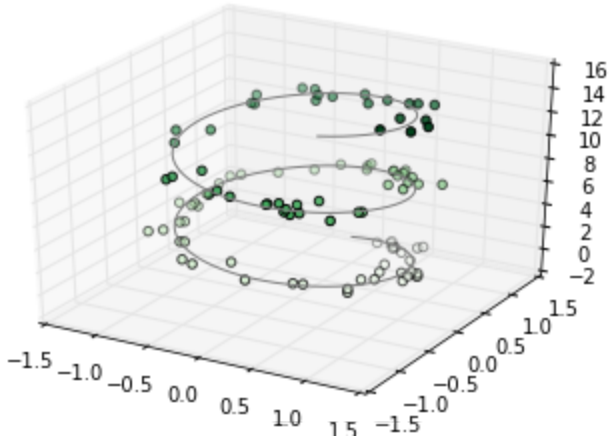


رسم نقاط و خطوط سه بعدی در پایتون

پایه ای‌ترین نمودار سه بعدی خط یا مجموعه ای از نقاط پراکنده است که با سه تایی (x,y,z) ایجاد می‌شود. این کار را با توابع ax.plot۳D و ax.scatter۳D انجام می‌دهیم. با کد زیر یک مارپیچ مثلثاتی (spiral) را به نقاطی که به طور تصادفی اطراف خط آن هستند، رسم می‌کنیم.

```
ax = plt.axes(projection='3d')
# Data for a three-dimensional line
zline = np.linspace(0, 15, 1000)
xline = np.sin(zline)
yline = np.cos(zline)
ax.plot3D(xline, yline, zline, 'gray')
# Data for three-dimensional scattered points
zdata = 15 * np.random.random(100)
xdata = np.sin(zdata) + 0.1 * np.random.randn(100)
ydata = np.cos(zdata) + 0.1 * np.random.randn(100)
ax.scatter3D(xdata, ydata, zdata, c=zdata, cmap='Greens');
```

خروجی به شکل زیر است:



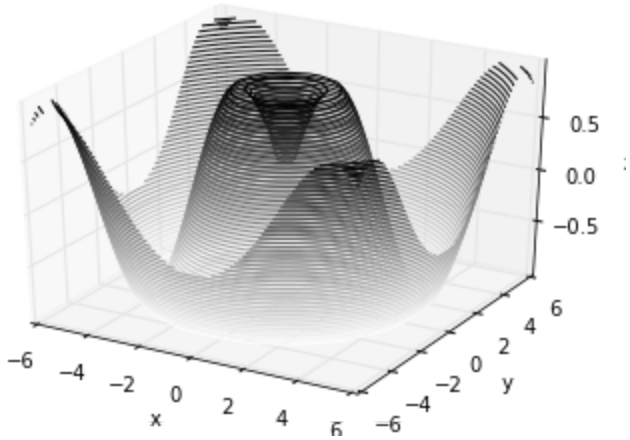
توجه کنید که به طور پیش فرض ، نقاط پراکندگی شفافیت خود را تنظیم می‌کنند تا حس عمق صفحه را نشان دهند.

رسم نمودارهای کانتور سه بعدی در پایتون

در مثال زیر یک نمودار کانتور سه بعدی از یک تابع سینوسی سه بعدی را نشان خواهیم داد.

```
def f(x, y):
    return np.sin(np.sqrt(x ** 2 + y ** 2))
x = np.linspace(-6, 6, 30)
y = np.linspace(-6, 6, 30)
X, Y = np.meshgrid(x, y)
Z = f(X, Y)
```

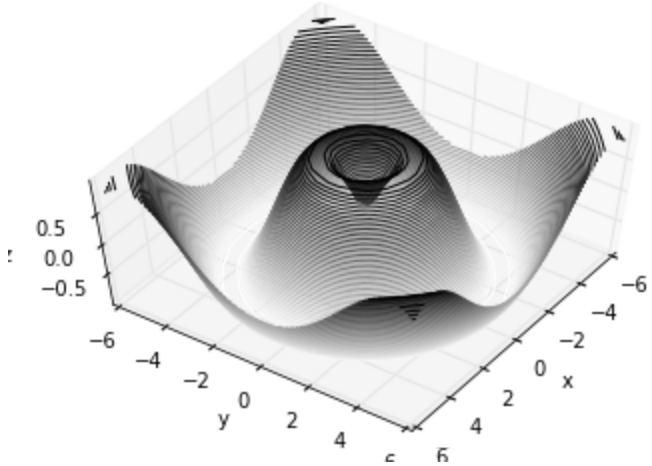
fig = plt.figure() ax = plt.axes(projection='۳d') ax.contour۳D(X, Y, Z, ۵۰, cmap='binary') ax.set_xlabel('x') ax.set_ylabel('y') ax.set_zlabel('z');خروجی، نموداری به شکل زیر است:



گاهی زاویه ی دید نمودار مناسب نیست. می‌توان با کمک تابع view_init زوایای ارتفاع و آزیموت نمودار را تنظیم کرد. در مثال زیر از ارتفاع ۶۰ درجه (یعنی ۶۰ درجه بالای صفحه ی x-y) و یک آزیموت ۳۵ درجه (یعنی ۳۵ درجه خلاف جهت عقربه‌های ساعت در مورد محور z می‌چرخیم) استفاده خواهیم کرد:

```
ax.view_init(60, 35)
fig
```

خروجی کد، نموداری به شکل زیر است:



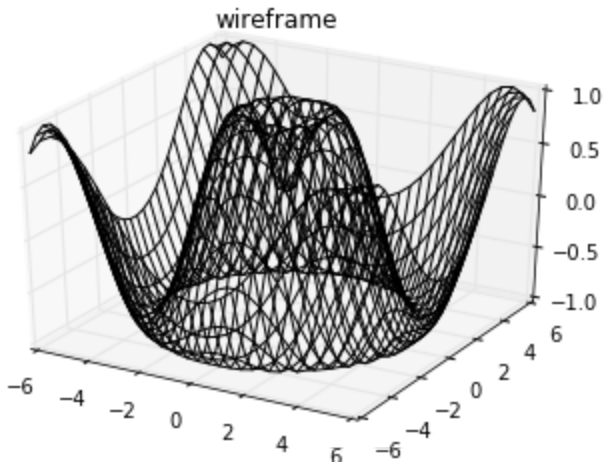
توجه داشته باشید که این نوع چرخش با استفاده از دکمه‌های تعاملی Matplotlib می‌تواند به صورت تعاملی با کلیک و کشیدن انجام شود.

قاب سیمی و پوسته ی سه بعدی در پایتون

دو نوع دیگر از اشکال سه بعدی که با داده‌های شبکه ای (grid) سر و کار دارند، قاب سیمی (wire frame) و پوسته (surface) نام دارند. این نمودارها شبکه ی داده‌ها را به شکل پوسته ای سه بعدی نمایش می‌دهند. در کد زیر مثالی از این نوع داده‌ها و ترسیمات را می‌بینیم:

```
fig = plt.figure()
ax = plt.axes(projection='3d')
ax.plot_wireframe(X, Y, Z, color='black')
ax.set_title('wireframe');
```

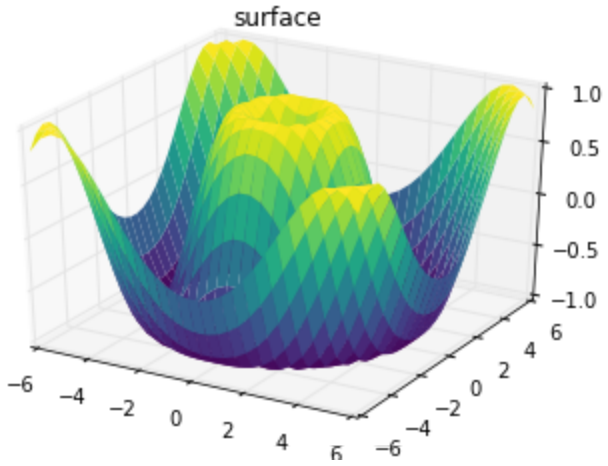
خروجی کد، به شکل زیر است:



طرح پوسته نیز مانند قاب سیمی است با این تفاوت که هر وجه آن با یک چند ضلعی پر شده است. افزودن یک طرح زنگی به هر وجه می‌تواند به تجسم و درک بهتر توپولوژی پوسته کمک کند.

```
ax = plt.axes(projection='3d')
ax.plot_surface(X, Y, Z, rstride=1, cstride=1,
                cmap='viridis', edgecolor='none')
ax.set_title('surface');
```

خروجی کد به شکل زیر است:



آموزش Python Seaborn

Seaborn کتابخانه ای برای ایجاد گرافیک های آماری در پایتون است. این کتابخانه بر مبنای matplotlib ساخته شده و با ساختار داده در Pandas ادغام شده است. برخی از کاربردهای seaborn عبارتند از:

API مبتنی بر مجموعه ی داده برای تست ارتباطات میان متغیرهای چندگانه

پشتیبانی ویژه برای متغیرهای طبقه بندی شده برای نمایش مشاهدات یا آمارها

گزینه هایی برای تجسم توزیع های تک متغیره یا دومتغیره و نیز مقایسه ی متغیرها میان زیرمجموعه هایی از داده

تخمین اتوماتیک و ترسیم مدل‌های رگرسیون خطی برای انواع مختلف متغیرهای وابسته

نماهای مناسب روی ساختار کلی مجموعه داده های پیچیده

انتزاعات سطح بالا برای ساختار شبکه‌های چند نموداری که به شما امکان ساخت تجسم های پیچیده‌تر را می‌دهد.

کنترل مختصر روی اشکال matplotlib با چندین تم از پیش ساخته شده

ابزاری برای انتخاب پالت‌های رنگی که الگوهای موجود در داده‌های شما را نشان می‌دهند.

هدف seaborn این است که تجسم سازی را به بخش اصلی کاوش و درک داده تبدیل کند. توابع نموداری مبتنی بر مجموعه ی داده بر روی دیتافریم‌ها و آرایه های حاوی مجموعه ی داده ها کار می‌کنند و به صورت داخلی نگاشت معنایی مورد نیاز و جمع-

آوری آماری را برای تولید نمودارهای دارای بار اطلاعاتی مفید انجام می‌دهد. برای اطلاعات بیشتر می‌توانید به **این سایت** مراجعه کنید.

تفاوت میان matplotlib و seaborn

در این قسمت این دو کتابخانه را با هم از نظر چند معیار بررسی می‌کنیم:

کارکرد:

Matplotlib: matplotlib به طور عمده برای ترسیمات پایه ای گسترش یافته است. مجسم سازی داده به طور عمده شامل نمودار میله ای، دایره ای، خطی، نمودارهای نقطه ای یا پراکنده و غیره می‌باشد.

Seaborn: seaborn از طرف دیگر الگوهای مختلفی از تجسم سازی را ارائه می دهد. این کتابخانه از قواعد دستوری کمتری استفاده می‌کند و تم‌های از پیش تعریف شده - ی جالبی دارد.

در تجسم سازی آماری بسیار متخصص است و در تجسم سازی داده های خلاصه و توزیع آماری داده‌ها به کار می رود.

کنترل اشکال چندگانه:

Matplotlib: matplotlib فیگورهای چندگانه را می‌تواند باز کند اما باید به طور صریح بسته شوند. تابع plt.close تنها فیگور فعلی را می بندد و plt.close('all') تمامی شکل ها را می بندد.

Seaborn: seaborn ایجاد اشکال چندگانه را به طور اتوماتیک ممکن می‌سازد ولی گاهی مشکلات ظرفیت حافظه ممکن است ایجاد شود.

تجسم سازی:

Matplotlib یک پکیج گرافیکی برای تجسم سازی داده در پایتون است. به خوبی با NumPy و Pandas تجمیع شده است. ماژول pyplot دستورات ترسیم MATLAB را منعکس می کند. از این رو کاربران متلب به راحتی در پایتون می‌توانند ترسیم داده بکنند.

Seaborn بیشتر با دیتافریم‌های پانداس تجمیع شده است. کتابخانه های matplotlib را برای ایجاد گرافیک های زیبا در پایتون با استفاده از متدهایی گسترش می دهد.

دیتافریم ها و آرایه ها:

Matplotlib با دیتافریم‌ها و آرایه ها کار می‌کند. APIهای متفاوتی برای ترسیم دارد. شکل‌ها توسط آبجکت هایی نمایش داده می شوند و تابع plot مشابه فراخوانی های بدون پارامتر است و نیاز به مدیریت پارامتر نیست.

Seaborn با مجموعه داده کار می کند و نسبت به matplotlib بسیار بصری تر است. در seaborn تابع replot برای مشخص کردن نوع نمودار ترسیمی مقدار پارامتر kind را مقداردهی می‌کند که مقدار این پارامتر می تواند بیانگر نوع خط، نمودار میله ای، هیستوگرام و... یا هر نوع دیگری از نمودارها باشد.

Seaborn برخلاف matplotlib حالت مند یا statefull نیست. از این رو تابع plot در آن نیاز به ارسال آبجکت دارد.

انعطاف پذیری:

Matplotlib بسیار قابل تنظیم و قدرتمند است.

Seaborn تم های از پیش آماده شده ی زیادی برای استفاده دارد.

موارد استفاده:

Pandas از matplotlib استفاده می کند. Seaborn برای موارد مورد استفاده ی خاص تری به کار می‌رود. تحت matplotlib است و بیشتر برای نمایش موارد آماری به کار می‌رود.

وارد کردن کتابخانه‌ها و مجموعه داده

کتابخانه ی pandas را که قابلیت بالایی در کنترل مجموعه داده‌های رابطه ای دارد وارد می‌کنیم سپس matplotlib را هم که ما را در افزودن ویژگی های مد نظرمان به نمودارها کمک می‌کند وارد می‌کنیم. سپس کتابخانه ی seaborn را import می کنیم. اکنون مجموعه داده ی مورد نظر را به شکل زیر وارد می‌کنیم:

```
# Pandas for managing datasets
import pandas as pd
# Matplotlib for additional customization
from matplotlib import pyplot as plt
# Seaborn for plotting and styling
import seaborn as sns
```

از مجموعه داده ی Pokemon استفاده می‌کنیم. این مجموعه داده را که به فرمت فایل csv است با کمک pandas به شکل دستور زیر می خوانیم:

```
# Read dataset
df = pd.read_csv('Pokemon.csv', index_col=0)
```

آرگومان index_col=۰ به این معنی است که ستون اول مجموعه داده را به عنوان ستون شناسه یا همان ID در نظر می‌گیریم. پنج سطر اول مجموعه داده به شکل زیر است:

	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Stage	Legendary
#												
1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	1	False
2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	2	False
3	Venusaur	Grass	Poison	525	80	82	83	100	100	80	3	False
4	Charmander	Fire	NaN	309	39	52	43	60	50	65	1	False
5	Charmeleon	Fire	NaN	405	58	64	58	80	65	80	2	False

همان طور که می‌بینید این دیتاست اطلاعات مربوط به آمار ۱۵۱ مورد Pokemon را دارد.

توابع ترسیم نمودار در seaborn

یکی از بزرگترین نقاط قوت seaborn تنوع و راحتی استفاده ی توابع موجود برای رسم نمودارها است. به طور مثال ایجاد نمودار پراکندگی (scatter plot) تنها با یک خط کد lmplot() است.

دو روش کلی که شما می توانید انجام دهید:

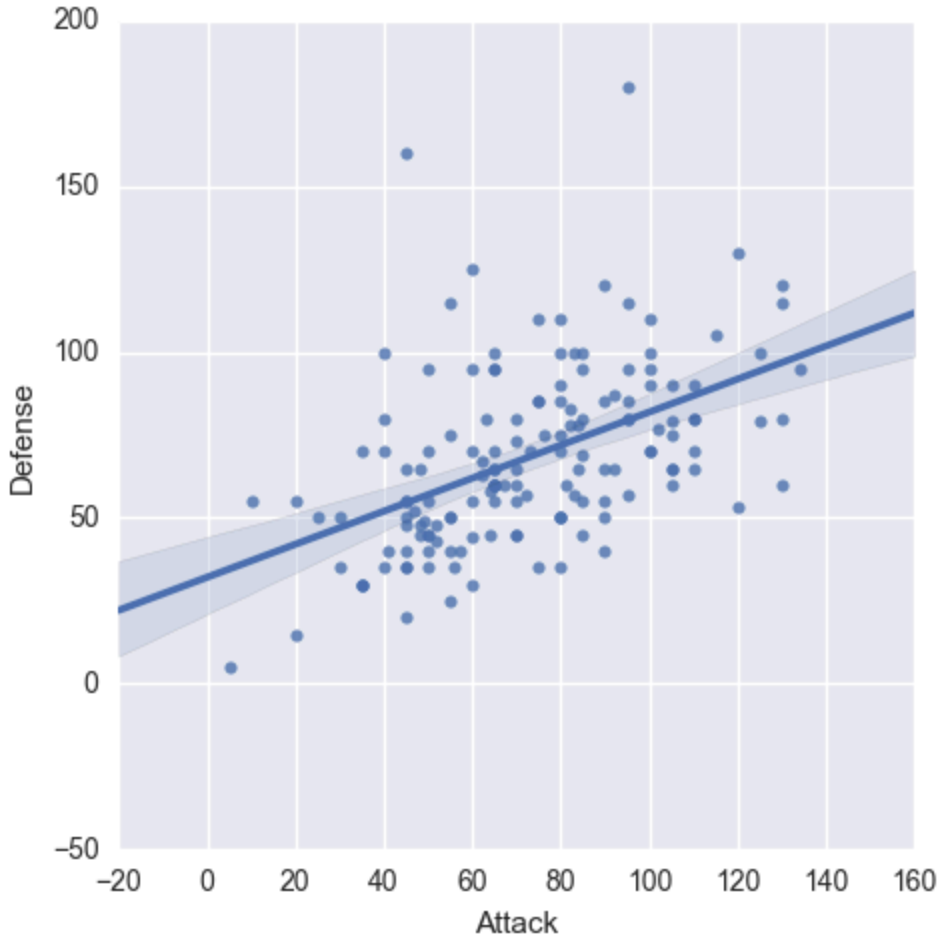
روش اول (پیشنهادی) ارسال دیتافریم با آرگومان data است و نام ستون‌ها به مقادیر x و y ارسال می شوند.

روش دوم ارسال مستقیم داده‌ها به شکل ساختار داده ی Series به آرگومان‌ها است.

به طور مثال در قطعه کد زیر ستون‌های Attack و Defense به دو روش بیان شده-اند:

```
# Recommended way
sns.lmplot(x='Attack', y='Defense', data=df)
# Alternative way
# sns.lmplot(x=df.Attack, y=df.Defense)
```

خروجی هردو کد، نقاطی به شکل زیر است(البته بدون خط مورب است نتیجه ی کد شما!)



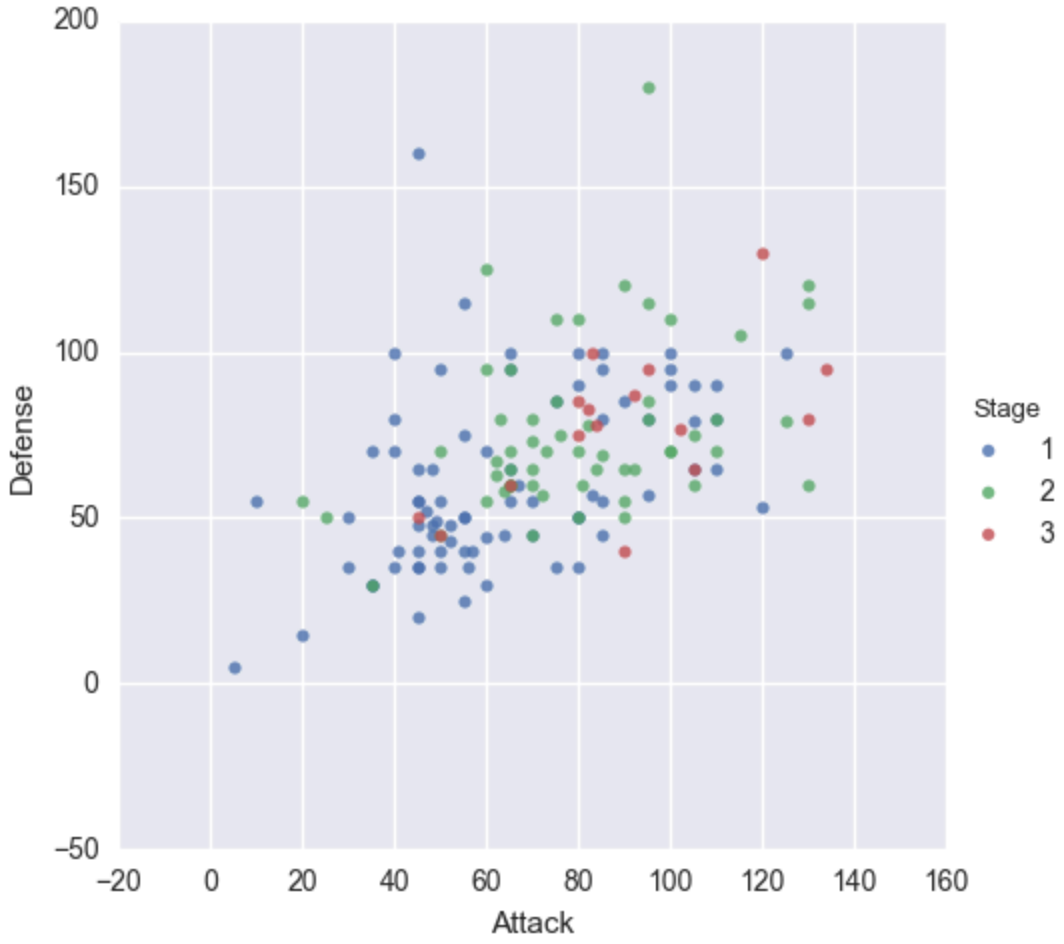
به هر حال seaborn تابعی برای رسم نمودار پراکندگی به طور اختصاصی ندارد از این رو در شکل یک خط مورب را مشاهده می کنید. (ما در واقع از تابع seaborn برای رسم رگرسیون خطی استفاده کرده ایم.)

خوشبختانه هر تابع رسم چندین گزینه ی مفید دارد که می‌توان بنا به نیاز آن ها را مقداردهی کرد. نحوه ی مقداردهی پارامترهای lmplot() به شکل زیر است:

مقدار fit_reg=False برای حذف رگرسیون خطی است چون تنها دنبال نمودار پراکندگی هستیم.

مقدار hue='Stage' برای رنگ آمیزی نقطه‌ها است. این آرگومان بسیار کاربردی است زیرا امکان نمایش بعد سومی از اطلاعات را به کمک رنگ نشان می‌دهد.

```
# Scatterplot arguments
sns.lmplot(x='Attack', y='Defense', data=df,
           fit_reg=False, # No regression line
           hue='Stage')   # Color by evolution stage
```



سفارشی سازی با Matplotlib

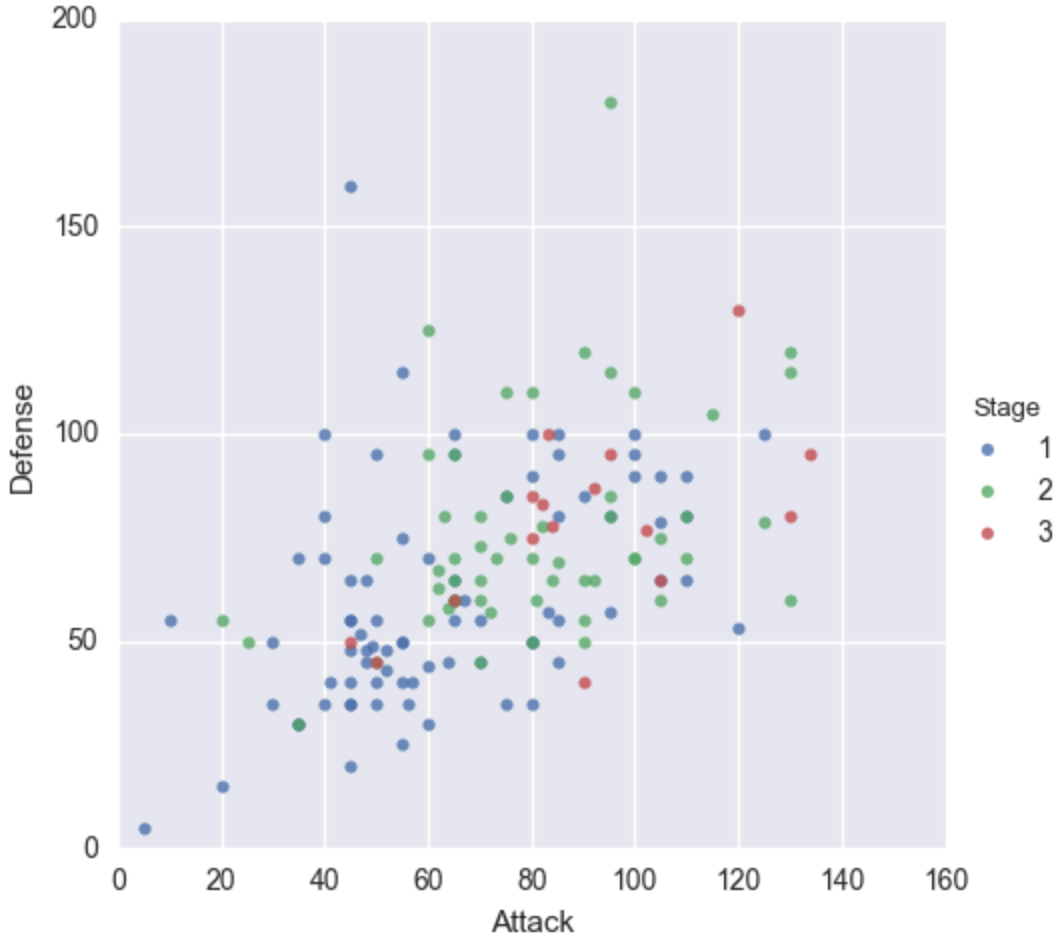
همان طور که قبلاً گفتیم seaborn بر روی matplotlib طراحی شده است و با اینکه امکانات بیشتری نسبت به matplotlib در اختیار ما قرار می‌دهد اما با این وجود گاهی به آن نیاز داریم. برای تنظیم محدوده ی محورها مراحل زیر را انجام دهید:

ابتدا تابع رسم در seaborn را به طور معمول فراخوانی کنید.

سپس تابع سفارشی سازی matplotlib را فراخوانی می‌کنیم. در این مثال از توابع ylim و xlim استفاده کرده ایم.

```
# Plot using Seaborn
sns.lmplot(x='Attack', y='Defense', data=df,
           fit_reg=False,
           hue='Stage')
# Tweak using Matplotlib
plt.ylim(0, None)
plt.xlim(0, None)
```

نتیجه ی حاصل به شکل زیر است:



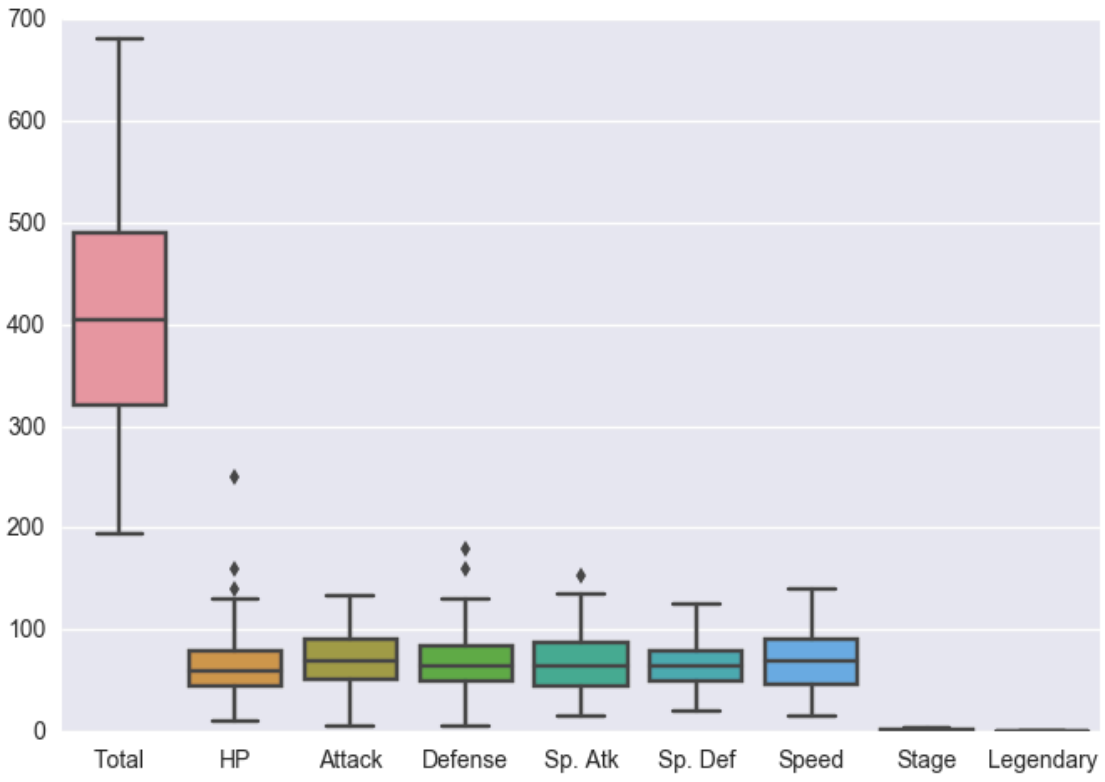
برای اطلاعات بیشتر در مورد توابع سفارشی سازی matplotlib به **داکیومنت** آن مراجعه کنید.

نقش کتابخانه ی Pandas در Seaborn

اگرچه در این مقاله به آموزش seaborn می‌پردازیم همان طور که پیش از این گفتیم pandas نقش مهمی دارد. توابع رسم نمودار seaborn از مفهوم ساختار داده ی دیتافریم بسیار بهره می‌برد. برای مثال در ادامه نحوه ی ایجاد نمودار جعبه ای را برای مجموعه داده مان بررسی می‌کنیم.

```
# Pandas for managing datasets
import pandas as pd
# Matplotlib for additional customization
from matplotlib import pyplot as plt
# Seaborn for plotting and styling
import seaborn as sns
# Read dataset
df = pd.read_csv("C:/Users/Naghsh/Downloads/Pokemon.csv",enc
#------------------------------------------------------------
# Boxplot
sns.boxplot(data=df)
```

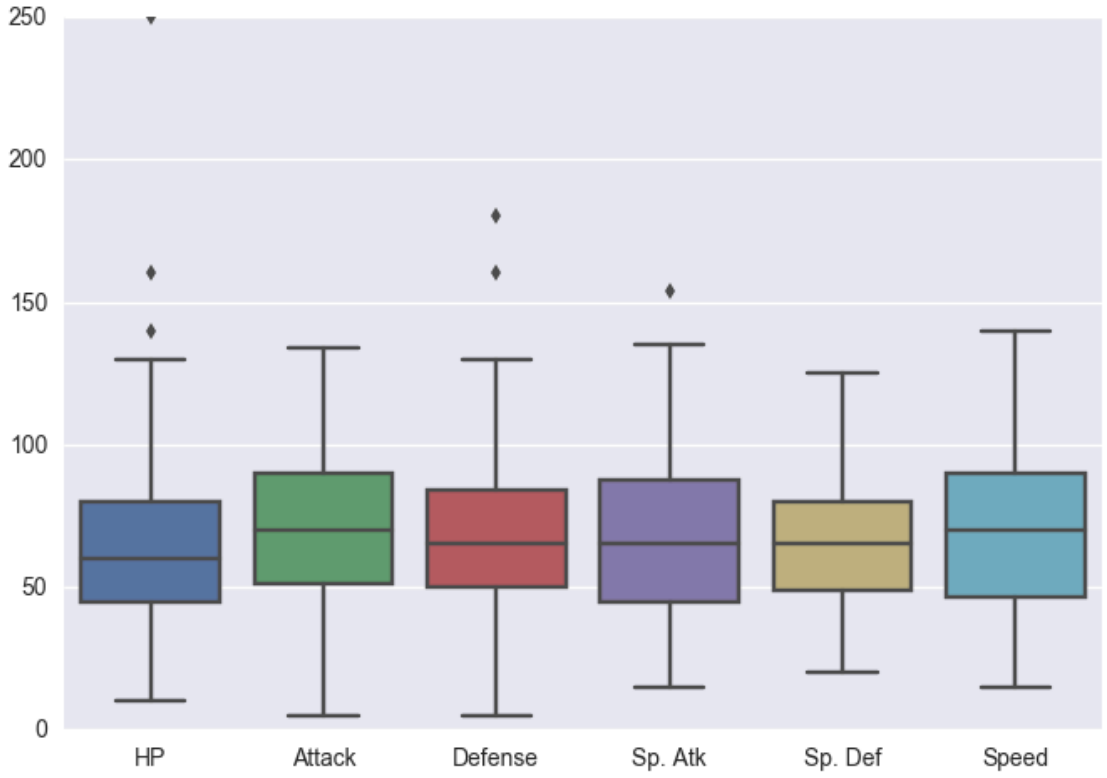
خروجی حاصل به شکل زیر است:



در این نمودار می‌توانیم ستون Total را حذف کنیم چون بقیه از هم مستقل اند، می‌توانیم Stage و Legendary را حذف کنیم چون تاثیری در آمار ندارند. به نظر می‌رسد که انجام این کار تنها به کمک seaborn کار ساده ای نیست. در عوض دیتافریم این کار را بسیار ساده می‌کند. بیایید دیتافریم جدیدی تحت عنوان stats_df که تنها ستون‌های آماری را نگه می‌دارد به شکل زیر تعریف کنیم.

```
# Pandas for managing datasets
import pandas as pd
# Matplotlib for additional customization
from matplotlib import pyplot as plt
# Seaborn for plotting and styling
import seaborn as sns
# Read dataset
df = pd.read_csv("C:/Users/Naghsh/Downloads/Pokemon.csv",enc
#------------------------------------------------------------
# Pre-format DataFrame
stats_df = df.drop(['Total', 'Stage', 'Legendary'], axis=1)
# New boxplot using stats_df
sns.boxplot(data=stats_df)
```

نمودار حاصل به شکل زیر است:



همان طور که در شکل می‌بینید به راحتی ستون‌های مورد نیاز را به کمک قواعد Pandas استخراج کردیم.

Seaborn : Seaborn themes تم‌های

مزیت دیگر seaborn وجود تم‌های شکل دهی مناسبی است که در آن گنجانده شده اند. تم پیش فرض "darkgrid" نام دارد. می‌توانیم این تم را حین ایجاد violin plot به 'whitegrid' تغییر دهیم.

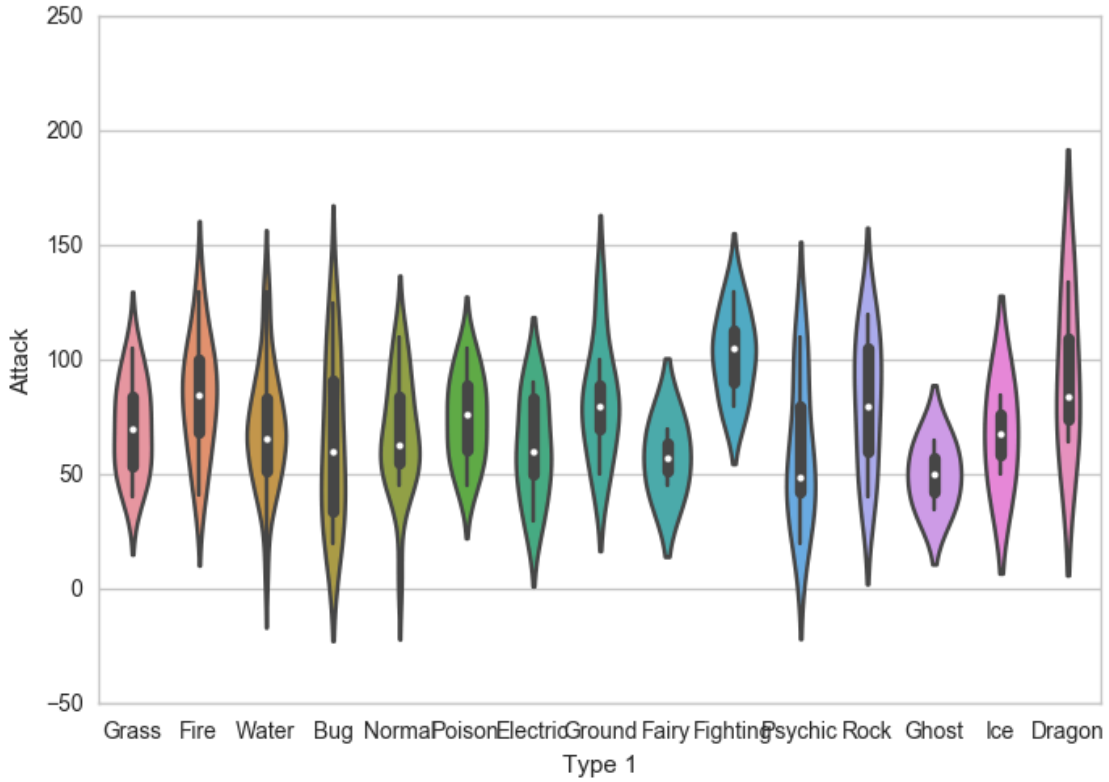
طرح violin جایگزین مناسبی برای نمودار جعبه ای است.

این نمودار توزیع داده را به کمک نمایش تغییر ضخامت عرض ویولن نشان می‌دهد. در حالی که در نمودار جعبه ای تنها خلاصه ای از اطلاعات آماری را می‌بینیم.

برای مثال می‌توانیم توزیع ستون Attack به شکل زیر نمایش دهیم:

```
# Pandas for managing datasets
import pandas as pd
# Matplotlib for additional customization
from matplotlib import pyplot as plt
# Seaborn for plotting and styling
import seaborn as sns
# Read dataset
df = pd.read_csv("C:/Users/Naghsh/Downloads/Pokemon.csv",enc
#------------------------------------------------------------
# Set theme
sns.set_style('whitegrid')
# Violin plot
sns.violinplot(x='Type 1', y='Attack', data=df)
```

خروجی حاصل به شکل زیر است:



همان طور که می‌بینید انواع Dragon تمایل دارند که آمار حملات بیشتری نسبت به انواع Ghost داشته باشند اما پراکندگی بزرگتری نیز دارند.

حالا طرفداران بازی Pokemon چیزهای جالبی راجب این نمودار ببینند: رنگ‌های به کار گرفته شده بدون مفهوم خاصی نیستند. چرا باید نوع Grass به رنگ صورتی یا داده ی Water به رنگ نارنجی باشد. بهتر است این را برطرف کنیم.

پالت‌های رنگ

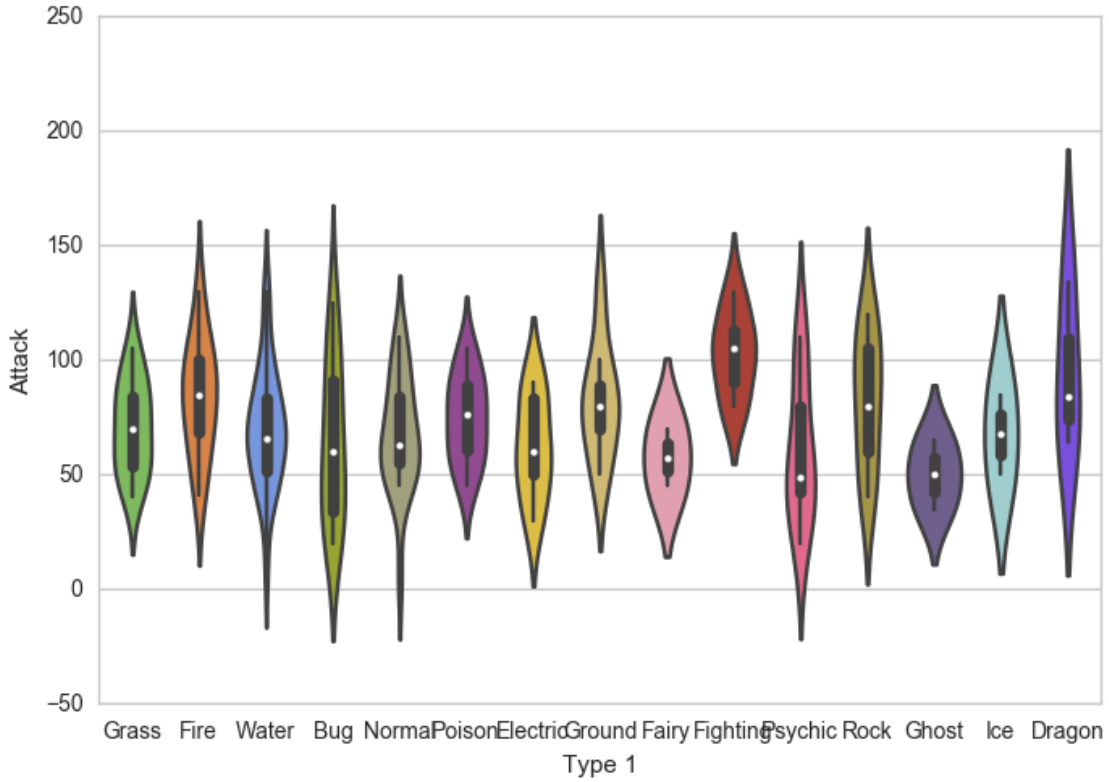
خوشبختانه seaborn امکان تنظیم پالت رنگی دلخواه را به ما می‌دهد. می توانیم لیستی از مقادیر کدهای هگزای رنگ را به سادگی ایجاد کنیم. در زیر لیستی از این رنگ‌ها را با هم می‌بینیم:

```
pkmn_type_colors = ['#78C850',  # Grass
                    '#F08030',  # Fire
                    '#6890F0',  # Water
                    '#A8B820',  # Bug
                    '#A8A878',  # Normal
                    '#A040A0',  # Poison
                    '#F8D030',  # Electric
                    '#E0C068',  # Ground
                    '#EE99AC',  # Fairy
                    '#C03028',  # Fighting
                    '#F85888',  # Psychic
                    '#B8A038',  # Rock
                    '#705898',  # Ghost
                    '#98D8D8',  # Ice
                    '#7038F8',  # Dragon
                   ]
```

به سادگی می‌توان با کمک آرگومان palette مقدار رنگ‌ها را دوباره برای نمودار خود تنظیم کنیم.

```
# Pandas for managing datasets
import pandas as pd
# Matplotlib for additional customization
from matplotlib import pyplot as plt
# Seaborn for plotting and styling
import seaborn as sns
# Read dataset
df = pd.read_csv("C:/Users/Naghsh/Downloads/Pokemon.csv",enc
#------------------------------------------------------------
pkmn_type_colors = ['#78C850',  # Grass
                    '#F08030',  # Fire
                    '#6890F0',  # Water
                    '#A8B820',  # Bug
                    '#A8A878',  # Normal
                    '#A040A0',  # Poison
                    '#F8D030',  # Electric
                    '#E0C068',  # Ground
                    '#EE99AC',  # Fairy
                    '#C03028',  # Fighting
                    '#F85888',  # Psychic
                    '#B8A038',  # Rock
                    '#705898',  # Ghost
                    '#98D8D8',  # Ice
                    '#7038F8',  # Dragon
                   ]
2
3
# Violin plot with Pokemon color palette
sns.violinplot(x='Type 1', y='Attack', data=df,
               palette=pkmn_type_colors) # Set color palette
```

خروجی حاصل به شکل زیر است:

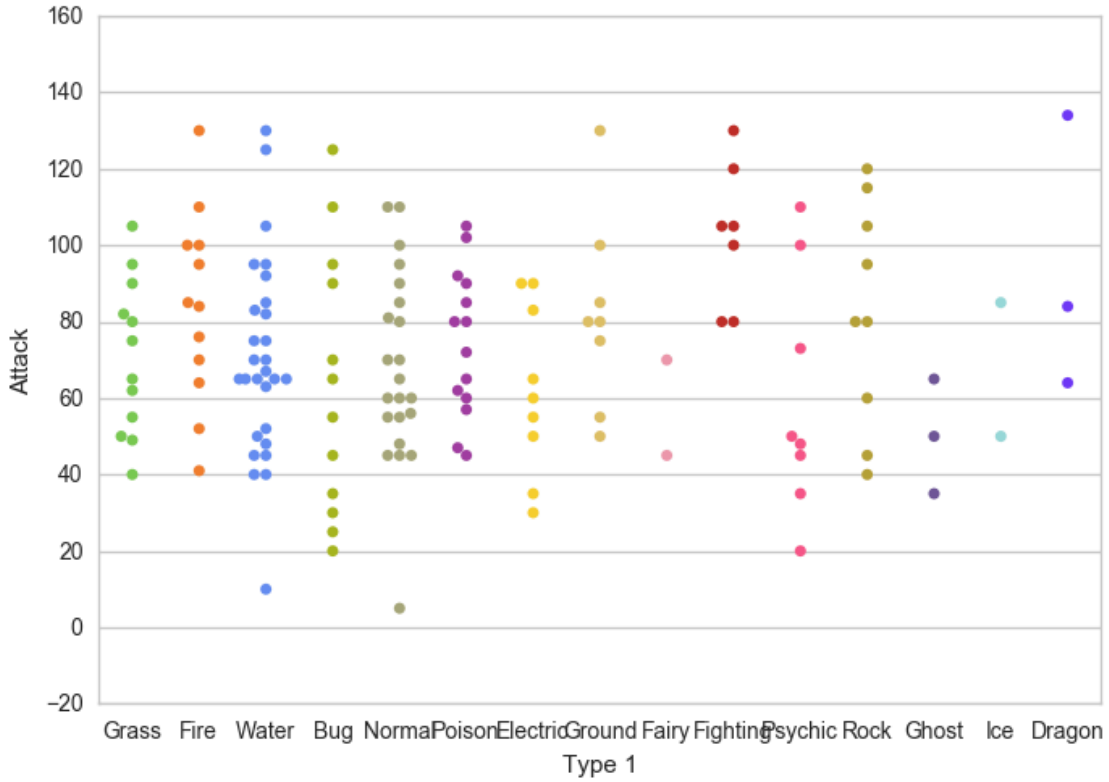


نمودارهای ویولن برای مصورسازی نحوه ی توزیع در آمار بسیار عالی هستند. به هر حال از آن جاییکه در مجموعه داده ی Pokemon تنها صد و پنجاه و یک نمونه داریم می‌توانیم به راحتی هر نمونه را نمایش دهیم. برای همین از نمودار پراکندگی (swarm plot) استفاده می‌کنیم.

شکل نهایی هر نمونه را به شکل نقطه ای مجزا با همان مقادیر در نمودار ویولن نشان می‌دهد.

```
# Pandas for managing datasets
import pandas as pd
# Matplotlib for additional customization
from matplotlib import pyplot as plt
# Seaborn for plotting and styling
import seaborn as sns
# Read dataset
df = pd.read_csv("C:/Users/Naghsh/Downloads/Pokemon.csv",enc
#-----------------------------------------------------------
pkmn_type_colors = ['#78C850',  # Grass
                    '#F08030',  # Fire
                    '#6890F0',  # Water
                    '#A8B820',  # Bug
                    '#A8A878',  # Normal
                    '#A040A0',  # Poison
                    '#F8D030',  # Electric
                    '#E0C068',  # Ground
                    '#EE99AC',  # Fairy
                    '#C03028',  # Fighting
                    '#F85888',  # Psychic
                    '#B8A038',  # Rock
                    '#705898',  # Ghost
                    '#98D8D8',  # Ice
                    '#7038F8',  # Dragon
                   ]
# Swarm plot with Pokemon color palette
sns.swarmplot(x='Type 1', y='Attack', data=df,
              palette=pkmn_type_colors)
```

خروجی حاصل به شکل زیر است:



حال سوال این است که آیا می‌توان نمودار swarm و ویولن را از آنجایی که اطلاعات مشابهی را نمایش می‌دهند، با هم ترکیب کرد؟

پاسخ مثبت است. پوشاندن نمودارها با یکدیگر به کمک seaborn ساده و مشابه matplotlib است. کاری که می‌بایست انجام دهیم به شرح زیر است:

در ابتدا شکل خود را به کمک matplotlib بزرگتر می‌کنیم.

سپس نمودار ویولن را رسم می‌کنیم و مقدار پارامتر inner را برابر None قرار می‌دهیم تا میله‌های داخل نمودار ویولن را حذف کنیم.

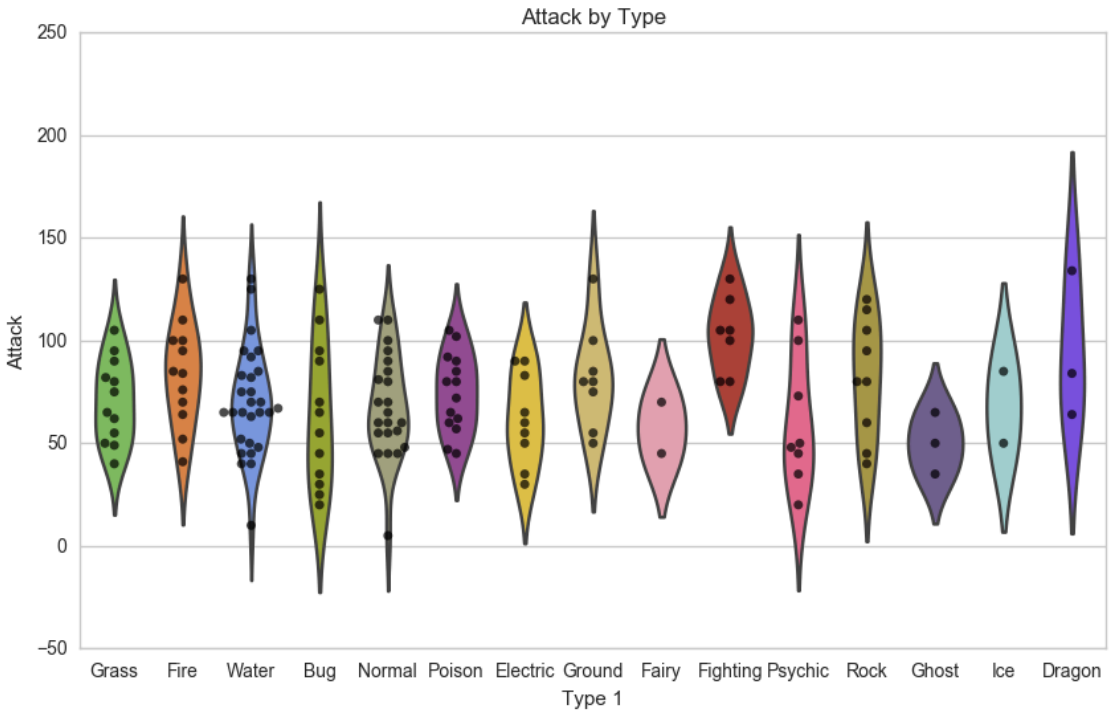
نمودار پراکندگی یا swarm را رسم می‌کنیم. این دفعه نقاط را سیاه رنگ ترسیم می‌کنیم.

در آخر عنوان نمودار را با matplotlib تنظیم می‌کنیم.

```
# Pandas for managing datasets
import pandas as pd
# Matplotlib for additional customization
from matplotlib import pyplot as plt
# Seaborn for plotting and styling
import seaborn as sns
# Read dataset
df = pd.read_csv("C:/Users/Naghsh/Downloads/Pokemon.csv",enc
#-------------------------------------------------------------
pkmn_type_colors = ['#78C850',  # Grass
                    '#F08030',  # Fire
                    '#6890F0',  # Water
                    '#A8B820',  # Bug
                    '#A8A878',  # Normal
                    '#A040A0',  # Poison
                    '#F8D030',  # Electric
                    '#E0C068',  # Ground
                    '#EE99AC',  # Fairy
                    '#C03028',  # Fighting
                    '#F85888',  # Psychic
                    '#B8A038',  # Rock
                    '#705898',  # Ghost
                    '#98D8D8',  # Ice
                    '#7038F8',  # Dragon
                   ]
# Set figure size with matplotlib
plt.figure(figsize=(10,6))
# Create plot
sns.violinplot(x='Type 1',
               y='Attack',
               data=df,
               inner=None, # Remove the bars inside the viol
               palette=pkmn_type_colors)
sns.swarmplot(x='Type 1',
              y='Attack',
              data=df,
              color='k', # Make points black
```

```
                alpha=0.7) # and slightly transparent
# Set title with matplotlib
plt.title('Attack by Type')
```

خروجی حاصل به شکل زیر است:



همان طور که در شکل می‌بینید نمودار جالبی داریم که چگونگی توزیع حملات را میان انواع مختلف pokemon نشان می‌دهد.

اما اگر بخواهیم سایر اطلاعات آماری را ببینیم بایستی چه کنیم؟

می توانیم نمودار فوق را برای سایر موارد آماری تکرار کنیم. اما راه دیگری وجود دارد که به کمک آن تمامی موارد را در یک نمودار نمایش دهیم.

تنها نیاز است که آماده سازی داده را به کمک pandas انجام دهیم. ابتدا شکل و نحوه ی داده‌ها در جدول مربوطه را یادآوری می‌کنیم.

```
# Pandas for managing datasets
import pandas as pd
# Matplotlib for additional customization
from matplotlib import pyplot as plt
# Seaborn for plotting and styling
import seaborn as sns
# Read dataset
df = pd.read_csv("C:/Users/Naghsh/Downloads/Pokemon.csv",enc
#------------------------------------------------------------
stats_df.head()
```

جدول داده مطابق شکل زیر است:

	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed
#									
1	Bulbasaur	Grass	Poison	45	49	49	65	65	45
2	Ivysaur	Grass	Poison	60	62	63	80	80	60
3	Venusaur	Grass	Poison	80	82	83	100	100	80
4	Charmander	Fire	NaN	39	52	43	60	50	65
5	Charmeleon	Fire	NaN	58	64	58	80	65	80

همان طور که می‌بینید تمامی آمارها در ستون‌های مجزا هستند. فرض کنید می‌خواهیم همه ی آن‌ها را در یک ستون ترکیب کنیم. به این منظور از تابع melt() پانداس استفاده می‌کنیم که سه پارامتر دارد.

دیتافریمی برای ترکیب داده ها

شناسه ی متغیرهایی که بدون تغییر باقی خواهند ماند.

در آخر نامی جدید برای متغیرهای ترکیب شده

خروجی به شکل زیر است:

```
# Pandas for managing datasets
import pandas as pd
# Matplotlib for additional customization
from matplotlib import pyplot as plt
# Seaborn for plotting and styling
import seaborn as sns
# Read dataset
df = pd.read_csv("C:/Users/Naghsh/Downloads/Pokemon.csv",enc
#------------------------------------------------------------
# Melt DataFrame
melted_df = pd.melt(stats_df,
                    id_vars=["Name", "Type 1", "Type 2"], #
                    var_name="Stat") # Name of melted variab
melted_df.head()
```

خروجی حاصل به شکل زیر است:

	Name	Type 1	Type 2	Stat	value
0	Bulbasaur	Grass	Poison	HP	45
1	Ivysaur	Grass	Poison	HP	60
2	Venusaur	Grass	Poison	HP	80
3	Charmander	Fire	NaN	HP	39
4	Charmeleon	Fire	NaN	HP	58

تمامی شش ستون آماری در یک ستون ترکیب شده اند و ستون آماری جدید بیانگر مقادیر اصلی است. در حقیقت اگر ابعاد این دو دیتافریم را مقایسه کنیم به نتیجه ی زیر می‌رسیم:

```
# Pandas for managing datasets
import pandas as pd
# Matplotlib for additional customization
from matplotlib import pyplot as plt
# Seaborn for plotting and styling
import seaborn as sns
# Read dataset
df = pd.read_csv("C:/Users/Naghsh/Downloads/Pokemon.csv",enc
#------------------------------------------------------------
# Melt DataFrame
melted_df = pd.melt(stats_df,
                    id_vars=["Name", "Type 1", "Type 2"], #
                    var_name="Stat") # Name of melted variab
print( stats_df.shape )
print( melted_df.shape )
```

ابعاد داده به شکل زیر است:

(۱۵۱, ۹) (۹۰۶, ۵)

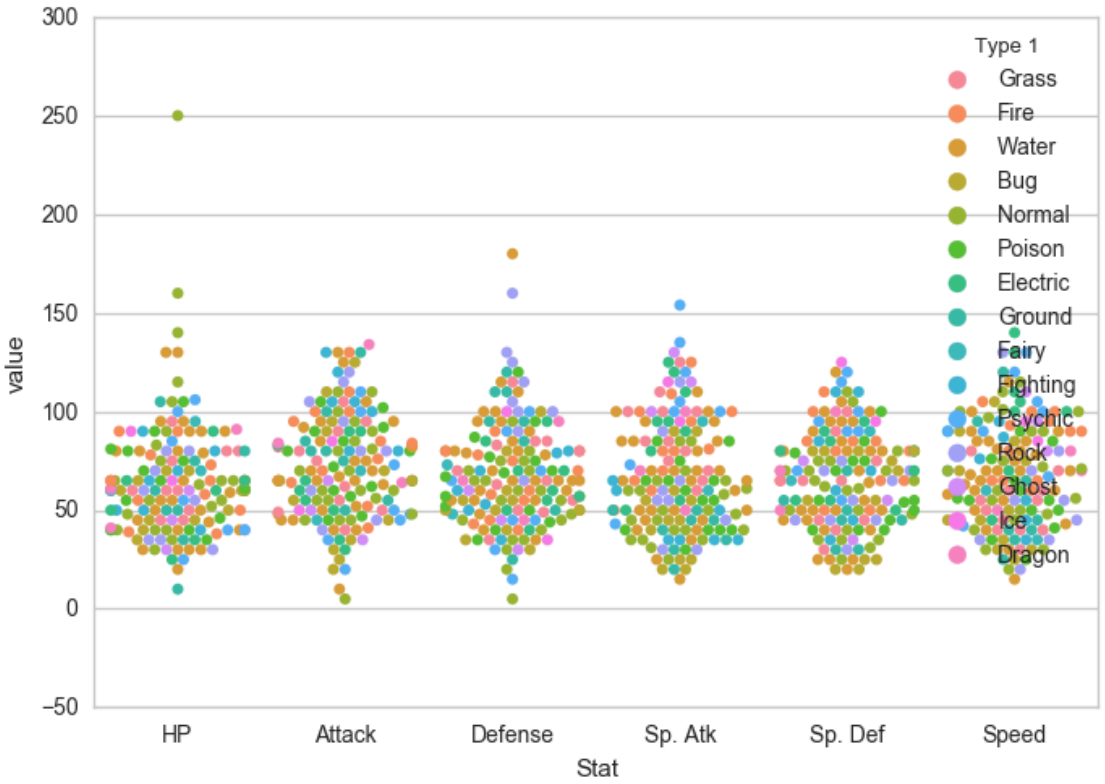
در اصل نکته اینجاست که melted_df شش مرتبه سطرهای stats_df را تکرار کرده است. می‌توانیم نمودار پراکندگی با melted_df را رسم کنیم. اما این دفعه:

مقادیر x و y را به شکل x=Stat و y=value تنظیم می‌کنیم و نقاط ما با توجه به مقادیر ستون‌ها پراکنده می‌شوند.

برای رنگ آمیزی بر اساس انواع pokemon مقدار پارامتر hue را برابر Type1 قرار می‌دهیم.

```
# Swarmplot with melted_df
sns.swarmplot(x='Stat', y='value', data=melted_df,
              hue='Type 1')
```

خروجی حاصل به شکل زیر است:



برای اینکه نمودار بالا خواناتر و بهتر شود می‌توان ترفندهای زیر را به کار برد:

بزرگ کردن نمودار

جدا کردن نقاط با کمک رنگ آمیزی و آرگومان split=True

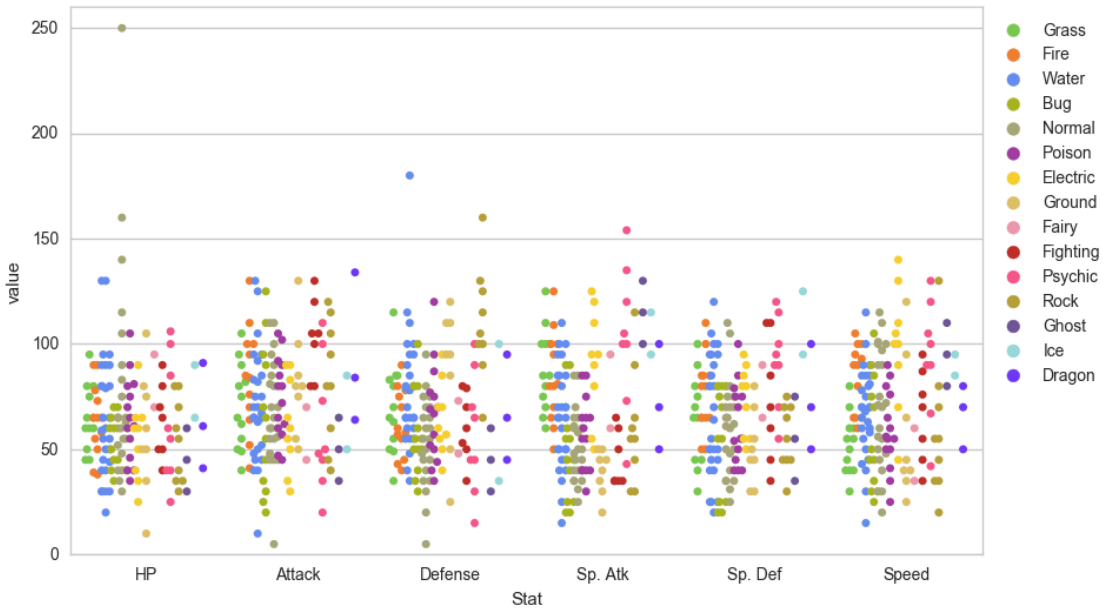
استفاده از پالت رنگی دلخواه

تنظیم محدوده ی محور y با شروع از نقطه ی صفر

قرار دادن راهنمای علایم و اختصارات یا همان legend در سمت راست

```
# 1. Enlarge the plot
plt.figure(figsize=(10,6))
sns.swarmplot(x='Stat',
              y='value',
              data=melted_df,
              hue='Type 1',
              split=True, # 2. Separate points by hue
              palette=pkmn_type_colors) # 3. Use Pokemon pal
# 4. Adjust the y-axis
plt.ylim(0, 260)
# 5. Place legend to the right
plt.legend(bbox_to_anchor=(1, 1), loc=2)
```

خروجی حاصل به شکل زیر است:

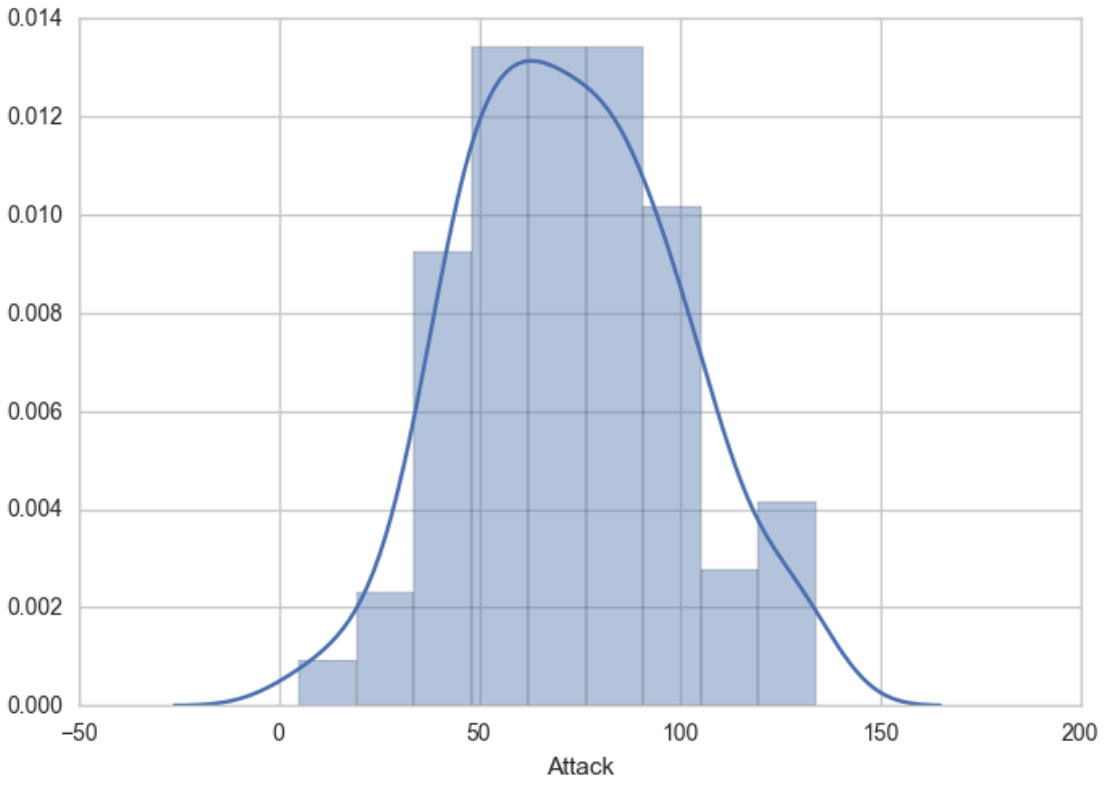


نمودار هیستوگرام در seaborn

نمودار هیستوگرام توزیع متغیرهای عددی را نمایش می‌دهد.

```
# Pandas for managing datasets
import pandas as pd
# Matplotlib for additional customization
from matplotlib import pyplot as plt
# Seaborn for plotting and styling
import seaborn as sns
# Read dataset
df = pd.read_csv("C:/Users/Naghsh/Downloads/Pokemon.csv",enc
#------------------------------------------------------------
# Distribution Plot (a.k.a. Histogram)
sns.distplot(df.Attack)
```

نمودار هیستوگرام برای نمایش چگونگی توزیع ستون Attack به شکل زیر است:

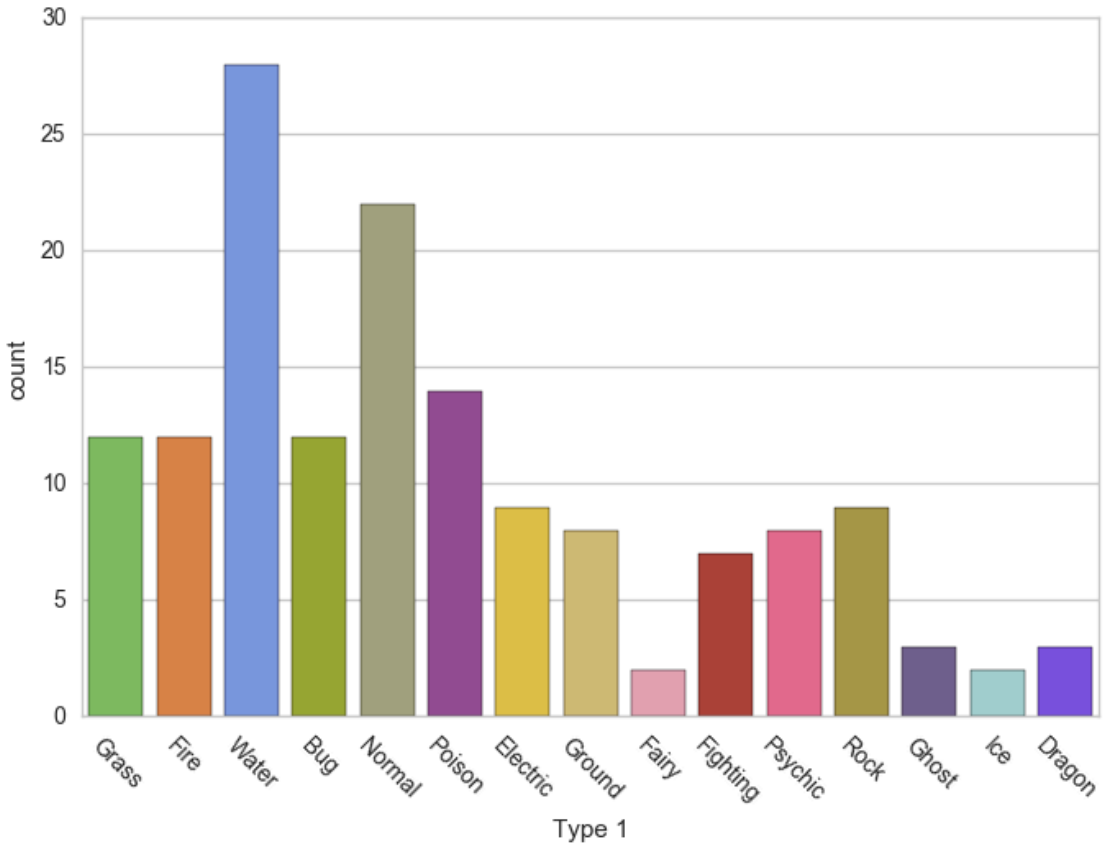


نمودار میله ای در seaborn

نمودار میله ای برای توزیع متغیرهای با مقادیر طبقه بندی شده مفید است.

```
# Count Plot (a.k.a. Bar Plot)
sns.countplot(x='Type 1', data=df, palette=pkmn_type_colors)
# Rotate x-labels
plt.xticks(rotation=-45)
```

خروجی حاصل به شکل زیر است:

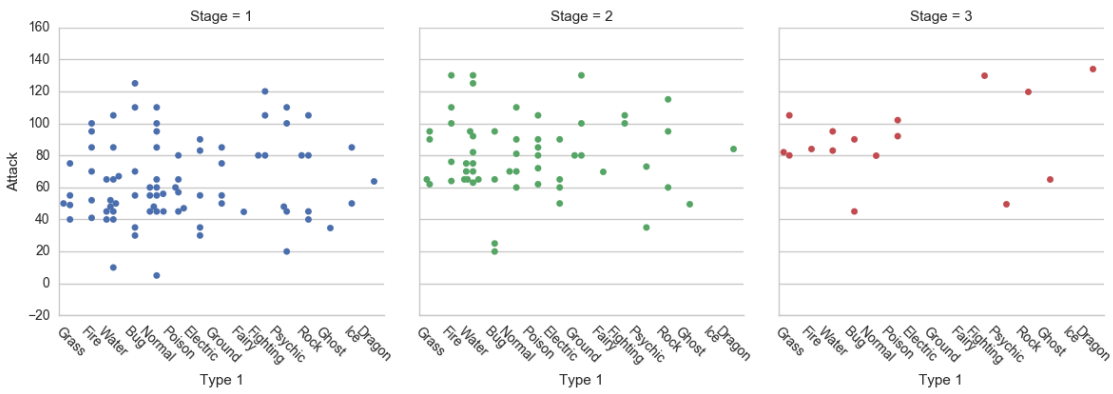


نمودار Factor در seaborn

نمودار فاکتور درک نقاط جدا از هم را با کلاس‌های طبقه بندی شده آسان می‌سازد

```
# Factor Plot
g = sns.factorplot(x='Type 1',
                   y='Attack',
                   data=df,
                   hue='Stage',  # Color by stage
                   col='Stage',  # Separate by stage
                   kind='swarm') # Swarmplot
# Rotate x-axis labels
g.set_xticklabels(rotation=-45)
# Doesn't work because only rotates last plot
# plt.xticks(rotation=-45)
```

خروجی حاصل به شکل زیر است:

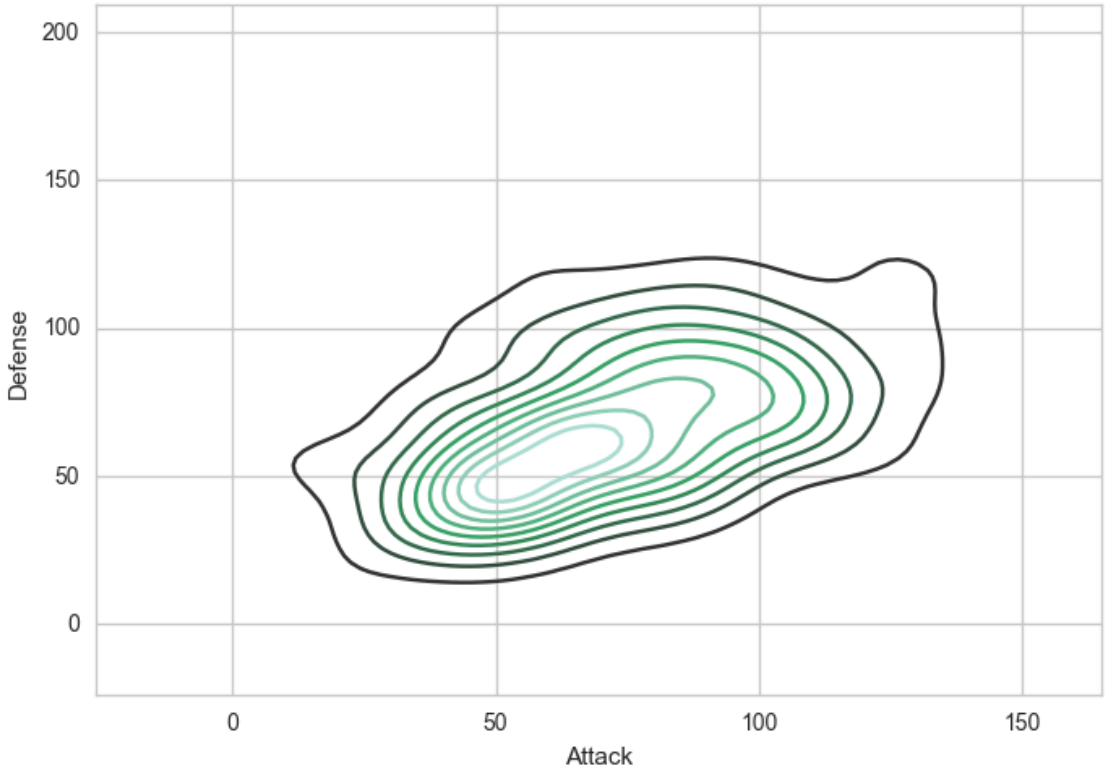


نمودار چگالی در seaborn

نمودار چگالی یا density plot چگونگی پراکندگی میان دو متغیر را نمایش می‌دهد.

```
# Density Plot
sns.kdeplot(df.Attack, df.Defense)
```

خروجی حاصل به شکل زیر است:

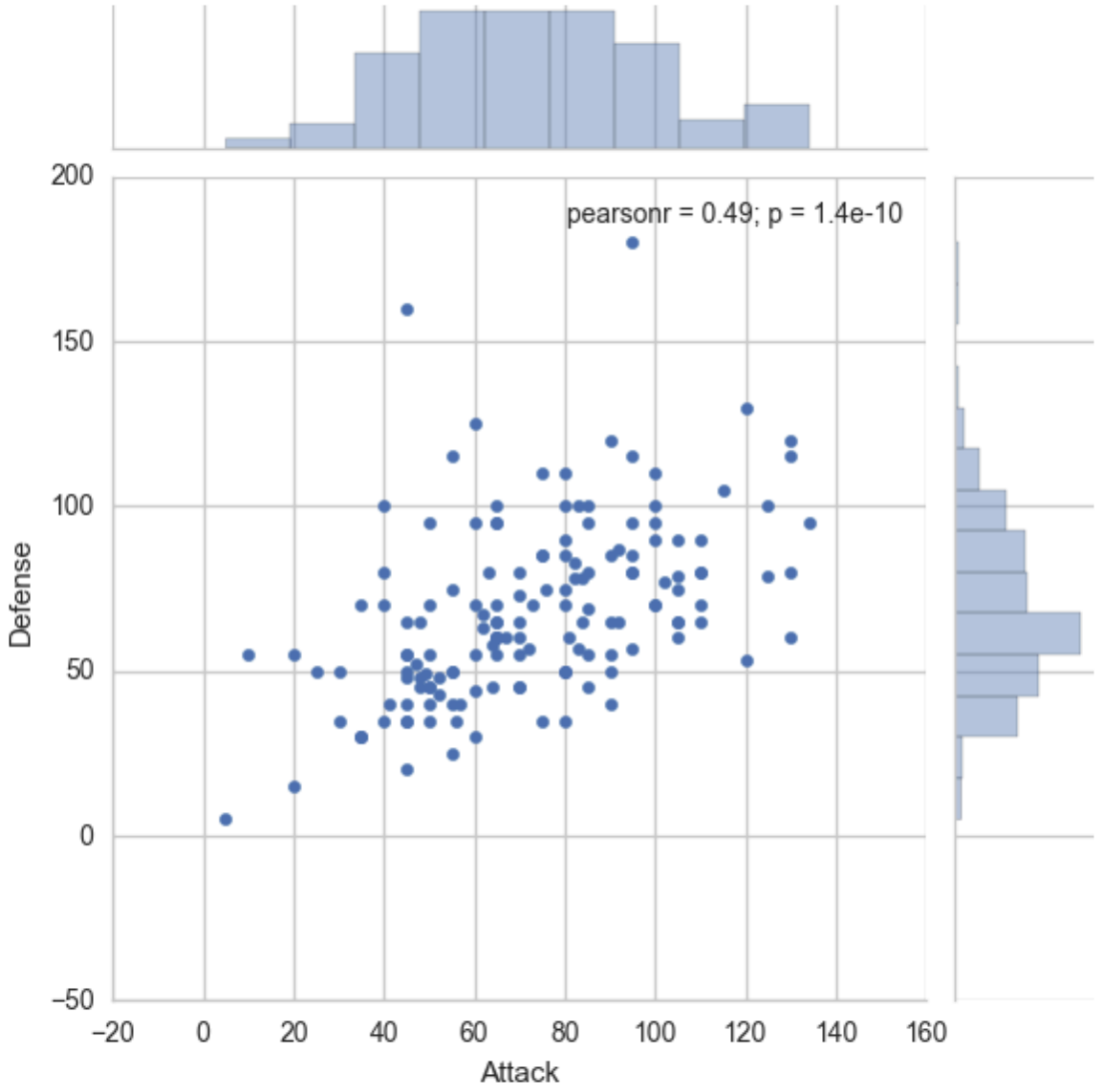


نمودارهای توزیعی متصل شده در seaborn

این نمودارها داده‌های میان نمودار پراکندگی و هیستوگرام را برای نمایش اطلاعات جزیی بر اساس توزیع‌های دو متغیره نشان می‌دهد.

```
# Joint Distribution Plot
sns.jointplot(x='Attack', y='Defense', data=df)
```

نتیجه ی حاصل توزیع فراوانی متغیرهای Attack و Defence و همچنین پراکندگی داده‌ها برحسب این دو متغیر را نمایش می‌دهد:





جمع بندی

کتابخانه‌های زیادی برای رسم نمودار در پایتون و مصور سازی داده‌ها موجود است. در این میان Mtplotlib و Seaborn جزو پرکاربردترین این کتابخانه‌ها هستند. Seaborn بر پایه ی Matplotlib بنا شده، ولی نسبت به آن قواعد دستوری کوتاه‌تر و تم‌های از پیش ساخته شده ای دارد. Matplotlib بیشتر برای رسم نمودارهای پایه ای به کار می‌رود و می‌توان تنظیمات دلخواه را بر روی نحوه ی نمایش نمودارها اعمال کرد.

انواع مختلف نمودارهای پرکاربرد و نحوه ی رسم آن‌ها با این دو پکیج در این مقاله آموزش داده شدند و شما می‌توانید با کمک آن‌ها نتایج خود را به راحتی مصورسازی و تحلیل کنید.

اگر دوست داری به یک متخصص داده کاوی اطلاعات با زبان پایتون تبدیل شوی و با استفاده از آن در بزرگترین شرکت‌ها مشغول به کار شوی، شرکت در **دوره دیتا ساینس** را پیشنهاد می‌کنیم.

مقالات مرتبط



پایتون

فتوشاپ آنلاین | معرفی ۳ ابزار کاربردی ویرایش تصاویر



پایتون

۱۷ سایت برای افزایش توانایی حل مسائل برنامه نویسی